

Human review packet: HT26EFXChores

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: HT26EFXChores
Paper id: HT26EFXChores
Source version: arXiv v2 (2026-07-09)
Source record: audit/paper_statement_map.json
Rows: 22
Generated: 2026-08-18
Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper HT26EFXChores --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local review trace.

Contents

- Semantic library prerequisites (26; not paper claims)
 - AdditiveChoreInstance
 - Bundle Item
 - Allocation Agent Item
 - EFXForChores
 - ChoreCost Agent Item
 - additiveChoreCost
 - EconCSLib.FairDivision.AdditiveChoreInstance.IsEFX
 - IsAllocationOf allocation chores
 - EconCSLib.FairDivision.AdditiveChoreInstance.IsFeasible
 - ParetoDominatesForChores
 - ParetoOptimalForChores
 - EconCSLib.FairDivision.AdditiveChoreInstance.IsParetoOptimal
 - EconCSLib.FairDivision.DoesNotStronglyEnvyForChores
 - EnvyFreeForChores
 - EconCSLib.FairDivision.IsBiValuedChoreCost
 - EconCSLib.FairDivision.IsSmallChore
 - ownSmallChoreSet
 - IsCanonicalSmallChoreAllocation
 - EconCSLib.FairDivision.IsLargeChore
 - IsOneOrRChoreCost
 - EconCSLib.FairDivision.smallAgentSet
 - EconCSLib.FairDivision.IsSmallForAtLeastThree
 - IsSmallForAtMostOne
 - EconCSLib.FairDivision.IsSmallForExactlyTwo
 - IsSuperCanonicalSmallChoreAllocation
 - EconCSLib.FairDivision.IsTriValuedChoreCost
- Main-text source claims (20)
 - Definition (EF for chores)

- Definition (EFX for chores)
- Definition (Pareto-Optimality)
- Definition (canonical allocation)
- Definition (canonical short/long labels)
- Definition (super-canonical allocation)
- Theorem 1 (tri-valued EFX nonexistence)
- Four-agent construction proposition: at most one A item per bundle
- Four-agent construction proposition: no-A bundle is expensive
- Theorem 2 (EFX and Pareto-optimality incompatibility)
- Theorem 2 proposition: every EFX agent receives a large item
- Theorem 2 proposition: EFX cost lower bounds
- Theorem 3 (four-agent bi-valued EFX existence)
- Lemma (M34 insertion)
- Lemma (composition)
- Lemma (canonical allocation properties)
- Lemma (balanced orientation)
- Lemma (EFX allocation of M2)
- Lemma (properties of the M2 EFX allocations)
- Lemma (exceptional-residue combination)
- Appendix and supplement source claims (2)
 - Appendix A proposition: no bundle contains two A items
 - Appendix A proposition: p1 and p2 lower bounds

Material library prerequisites

AdditiveChoreInstance

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:244-248

Verbatim paper-source connection

```
\section{Preliminaries}
A set  $M$  of  $m$  chores  $g_1, \dots, g_m$  is allocated to a set  $N$  of  $n$  agents  $1, \dots, n$ .
We will use item and chore interchangeably in this paper to refer to an element in  $M$ .
Each agent  $i$ 's cost function  $c_i: 2^M \rightarrow \mathbb{R}_{\geq 0}$  is additive:
 $c_i(S) = \sum_{g \in S} c_i(g)$ .
```

Lean declaration type (metadata)

Type $u_3 \rightarrow \text{Type } u_4 \rightarrow \text{Type } (\max u_3 u_4)$

Exact Lean library declaration

```
-- An additive indivisible-chore instance: a finite chore set with nonnegative
per-agent item costs. -/
structure AdditiveChoreInstance (Agent Item : Type*) where
  -- The chores that must be allocated. -/
  chores : Finset Item
  -- The cost of each individual chore for each agent. -/
  cost : ChoreCost Agent Item
  -- Source-model nonnegativity of individual chore costs. -/
  nonneg : ∀ agent item, 0 ≤ cost agent item
```

Recorded source-to-library screening Verdict: matches

Reason: The structure packages the source's finite chore set and nonnegative pointwise costs. The separate additiveChoreCost review checks the displayed finite-sum extension.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Bundle Item

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:244-248

Verbatim paper-source connection

```
\section{Preliminaries}
A set  $M$  of  $m$  chores  $g_1, \dots, g_m$  is allocated to a set  $N$  of  $n$  agents  $1, \dots, n$ .
We will use `item' and `chore' interchangeably in this paper to refer to an element in  $M$ .
Each agent  $i$ 's cost function  $c_i: 2^M \rightarrow \mathbb{R}_{\geq 0}$  is additive:
 $c_i(S) = \sum_{g \in S} c_i(g)$ .
```

Lean-expanded library semantic target

```
(Item : Type u_1) → Finset Item
```

Exact Lean library declaration

```
-- A bundle of indivisible goods. -/
abbrev Bundle (Item : Type*) := Finset Item
```

Recorded source-to-library screening Verdict: matches

Reason: The source introduces a finite chore set and identifies an item or chore with an element of that set. The Lean abbreviation is the corresponding finite bundle representation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Allocation Agent Item

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:244-248

Verbatim paper-source connection

```
\section{Preliminaries}
A set  $M$  of  $m$  chores  $g_1, \dots, g_m$  is allocated to a set  $N$  of  $n$  agents  $1, \dots, n$ .
We will use 'item' and 'chore' interchangeably in this paper to refer to an element in  $M$ .
Each agent  $i$ 's cost function  $c_i: 2^M \rightarrow \mathbb{R}_{\geq 0}$  is additive:
 $c_i(S) = \sum_{g \in S} c_i(g)$ .
```

Lean-expanded library semantic target

```
(Agent : Type u_1) → (Item : Type u_2) → Agent → EconCSLib.FairDivision.Bundle Item
```

Exact Lean library declaration

```
-- An allocation assigns each agent a bundle. -/
abbrev Allocation (Agent Item : Type*) := Agent → Bundle Item
```

Recorded source-to-library screening Verdict: matches

Reason: The source writes an allocation as a tuple of one bundle for each agent. The Lean definition is that agent-indexed tuple; complete allocation is reviewed separately.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EFXForChores

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:260-275

Verbatim paper-source connection

```
\begin{definition}[EFX for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{EFX} if for every pair of agents  $i,j$  we have either
\begin{itemize}
\item  $X_i=\emptyset$ , or
\item for every item  $g\in X_i$ ,  $c_i(X_i\setminus\{g\})\leq c_i(X_j)$ .
\end{itemize}
Equivalently, if
\l[
\tau_i(X):=\tau_i(X_i):=
\begin{cases}
0, & \text{if } X_i=\emptyset, \\
c_i(X_i)-\min_{g\in X_i}c_i(g), & \text{if } X_i\neq\emptyset,
\end{cases}
\l]
then  $X$  is EFX if and only if  $\tau_i(X)\leq c_i(X_j)$  for all  $i,j$ .
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
(allocation : EconCSLib.FairDivision.Allocation Agent Item) (i j : Agent),
allocation i = ∅ ∨ ∀ item ∈ allocation i, cost i (allocation i \ {item}) ≤ cost i (allocation j)
```

Exact Lean library declaration

```
-- Envy-freeness up to any item for chores: after removing any item from an
agent's own bundle, that agent incurs no more cost than for another bundle. -/
def EFXForChores [DecidableEq Item] (cost : Agent → Bundle Item → ℝ)
(allocation : Allocation Agent Item) : Prop :=
∀ i j, allocation i = ∅ ∨
∀ item ∈ allocation i, cost i (allocation i \ {item}) ≤ cost i (allocation j)
```

Recorded source-to-library screening Verdict: matches

Reason: The Lean declaration has the same empty-own-bundle alternative and universal one-item-removal inequality as the displayed EFX definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ChoreCost Agent Item

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:244-248

Verbatim paper-source connection

```
\section{Preliminaries}
A set  $M$  of  $m$  chores  $g_1, \dots, g_m$  is allocated to a set  $N$  of  $n$  agents  $1, \dots, n$ .
We will use 'item' and 'chore' interchangeably in this paper to refer to an element in  $M$ .
Each agent  $i$ 's cost function  $c_i: 2^M \rightarrow \mathbb{R}_{\geq 0}$  is additive:
 $c_i(S) = \sum_{g \in S} c_i(g)$ .
```

Lean-expanded library semantic target

```
(Agent : Type u_3) → (Item : Type u_4) → Agent → Item → ℝ
```

Exact Lean library declaration

```
-- Per-agent costs of individual chores. -/
abbrev ChoreCost (Agent Item : Type*) := Agent → Item → ℝ
```

Recorded source-to-library screening Verdict: matches

Reason: This is the source's pointwise component $c_i(g)$ of the displayed additive cost function. Nonnegativity and finite-sum extension are reviewed by the corresponding instance and additive-cost definitions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

additiveChoreCost

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:244-248

Verbatim paper-source connection

```
\section{Preliminaries}
A set  $M$  of  $m$  chores  $g_1, \dots, g_m$  is allocated to a set  $N$  of  $n$  agents  $1, \dots, n$ .
We will use 'item' and 'chore' interchangeably in this paper to refer to an element in  $M$ .
Each agent  $i$ 's cost function  $c_i: 2^M \rightarrow \mathbb{R}_{\geq 0}$  is additive:
 $c_i(S) = \sum_{g \in S} c_i(\{g\})$ .
```

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
(cost : EconCSLib.FairDivision.ChoreCost Agent Item) →
(agent : Agent) → (bundle : EconCSLib.FairDivision.Bundle Item) →  $\sum$  item  $\in$  bundle, cost agent item
```

Exact Lean library declaration

```
-- The additive cost of a finite bundle of chores. -/
noncomputable def additiveChoreCost (cost : ChoreCost Agent Item)
  (agent : Agent) (bundle : Bundle Item) :  $\mathbb{R}$  :=
  bundle.sum fun item => cost agent item
```

Recorded source-to-library screening Verdict: matches

Reason: The Lean definition sums the source's singleton costs over the finite bundle, exactly as the displayed additive-cost equation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.AdditiveChoreInstance.IsEFX

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:260-275

Verbatim paper-source connection

```
\begin{definition}[EFX for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{EFX} if for every pair of agents  $i,j$  we have either
\begin{itemize}
\item  $X_i=\emptyset$ , or
\item for every item  $g\in X_i$ ,  $c_i(X_i\setminus\{g\})\leq c_i(X_j)$ .
\end{itemize}
Equivalently, if
\[[\tau_i(X):=\tau_i(X_i)=0, \& X_i=\emptyset, \& c_i(X_i)-\min_{g\in X_i}c_i(g), \& X_i\neq\emptyset,
\end{cases}
\]
then  $X$  is EFX if and only if  $\tau_i(X)\leq c_i(X_j)$  for all  $i,j$ .
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : DecidableEq Item]
(choreInstance : EconCSLib.FairDivision.AdditiveChoreInstance Agent Item)
(allocation : EconCSLib.FairDivision.Allocation Agent Item),
EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost choreInstance.cost) allocation
```

Exact Lean library declaration

```
def IsEFX {Agent Item : Type*} [DecidableEq Item]
(choreInstance : AdditiveChoreInstance Agent Item)
(allocation : Allocation Agent Item) : Prop :=
EFXForChores (additiveChoreCost choreInstance.cost) allocation
```

Recorded source-to-library screening Verdict: matches

Reason: This is the source EFX predicate applied to the instance's additive cost function. The exact empty-bundle alternative and one-item-removal inequality are independently exposed by EFXForChores.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

IsAllocationOf allocation chores

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:244-248

Verbatim paper-source connection

```
\section{Preliminaries}
A set  $M$  of  $m$  chores  $g_1, \dots, g_m$  is allocated to a set  $N$  of  $n$  agents  $1, \dots, n$ .
We will use ' $\text{item}$ ' and ' $\text{chore}$ ' interchangeably in this paper to refer to an element in  $M$ .
Each agent  $i$ 's cost function  $c_i: 2^M \rightarrow \mathbb{R}_{\geq 0}$  is additive:
 $c_i(S) = \sum_{g \in S} c_i(\{g\})$ .
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [DecidableEq Item] (A : EconCSLib.FairDivision.Allocation Agent Item)
  (goods : Finset Item), (∀ (i : Agent), ∀ g ∈ A i, g ∈ goods) ∧ ∀ g ∈ goods, ∃! i, g ∈ A i
```

Exact Lean library declaration

```
/-- `A` allocates exactly the goods in `goods`, with no duplicates and no outsiders. -/
def IsAllocationOf [DecidableEq Item] (A : Allocation Agent Item) (goods : Finset Item) : Prop :=
  (∀ i g, g ∈ A i → g ∈ goods) ∧
  ∀ g, g ∈ goods → ∃! i, g ∈ A i
```

Recorded source-to-library screening Verdict: matches

Reason: The source allocation convention assigns the finite chore set to the agents. The Lean predicate makes that convention explicit by requiring every chore exactly once and no outside item.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.AdditiveChoreInstance.IsFeasible

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:260-275

Verbatim paper-source connection

```
\begin{definition}[EFX for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{EFX} if for every pair of agents  $i,j$  we have either
\begin{itemize}
\item  $X_i=\emptyset$ , or
\item for every item  $g\in X_i$ ,  $c_i(X_i\setminus\{g\})\leq c_i(X_j)$ .
\end{itemize}
\end{definition}
Equivalently, if
\[[\tau_i(X) := \tau_i(X_i) = 0, \text{ \& } X_i = \emptyset, \\ c_i(X_i) - \min_{g \in X_i} c_i(g), \text{ \& } X_i \neq \emptyset, \\ \end{cases}]
\]
then  $X$  is EFX if and only if  $\tau_i(X) \leq c_i(X_j)$  for all  $i,j$ .
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : DecidableEq Item]
(choreInstance : EconCSLib.FairDivision.AdditiveChoreInstance Agent Item)
(allocation : EconCSLib.FairDivision.Allocation Agent Item),
EconCSLib.FairDivision.IsAllocationOf allocation choreInstance.chores
```

Exact Lean library declaration

```
def IsFeasible {Agent Item : Type*} [DecidableEq Item]
(choreInstance : AdditiveChoreInstance Agent Item)
(allocation : Allocation Agent Item) : Prop :=
IsAllocationOf allocation choreInstance.chores
```

Recorded source-to-library screening Verdict: matches

Reason: The paper's allocation convention allocates the chore set among agents. This predicate specializes the separately reviewed complete-allocation condition to the instance's declared chore set; it adds no source-side feasibility condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ParetoDominatesForChores

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:280-287

Verbatim paper-source connection

```
\begin{definition}[Pareto-Optimality]
  An allocation  $Y=(Y_1,\ldots,Y_n)$  \emph{Pareto-dominates} an allocation  $X=(X_1,\ldots,X_n)$  if
  \begin{itemize}
    \item  $c_i(Y_i) \leq c_i(X_i)$  holds for all  $i \in \{1,\ldots,n\}$ , and
    \item there exists  $i \in \{1,\ldots,n\}, \wedge; c_i(Y_i) < c_i(X_i)$ .
  \end{itemize}
  An allocation  $X=(X_1,\ldots,X_n)$  is \emph{Pareto-optimal} if it is not Pareto-dominated by any allocation.
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
  (allocation improvement : EconCSLib.FairDivision.Allocation Agent Item),
  (∀ (i : Agent), cost i (improvement i) ≤ cost i (allocation i)) ∧ ∃ i, cost i (improvement i) < cost i (allocation i)
```

Exact Lean library declaration

```
/-- Allocation `improvement` Pareto-dominates `allocation` for chores when it
weakly decreases every cost and strictly decreases one cost. -/
def ParetoDominatesForChores (cost : Agent → Bundle Item → ℝ)
  (allocation improvement : Allocation Agent Item) : Prop :=
  (∀ i, cost i (improvement i) ≤ cost i (allocation i)) ∧
  ∃ i, cost i (improvement i) < cost i (allocation i)
```

Recorded source-to-library screening Verdict: matches

Reason: The displayed Pareto-dominance definition says that an improvement weakly decreases every agent's cost and strictly decreases one cost. The Lean declaration is exactly that two-part condition for an improvement and a current allocation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ParetoOptimalForChores

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:280-287

Verbatim paper-source connection

```
\begin{definition}[Pareto-Optimality]
  An allocation  $Y=(Y_1,\ldots,Y_n)$  \emph{Pareto-dominates} an allocation  $X=(X_1,\ldots,X_n)$  if
  \begin{itemize}
    \item  $c_i(Y_i) \leq c_i(X_i)$  holds for all  $i \in \{1,\ldots,n\}$ , and
    \item there exists  $i \in \{1,\ldots,n\}, \wedge; c_i(Y_i) < c_i(X_i)$ .
  \end{itemize}
  An allocation  $X=(X_1,\ldots,X_n)$  is \emph{Pareto-optimal} if it is not Pareto-dominated by any allocation.
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
(chores : Finset Item) (allocation : EconCSLib.FairDivision.Allocation Agent Item),
¬∃ improvement,
  EconCSLib.FairDivision.IsAllocationOf improvement chores ∧
  EconCSLib.FairDivision.ParetoDominatesForChores cost allocation improvement
```

Exact Lean library declaration

```
-- A complete chore allocation is Pareto-optimal if no other complete
allocation Pareto-dominates it in the cost direction. -/
def ParetoOptimalForChores [DecidableEq Item] (cost : Agent → Bundle Item → ℝ)
  (chores : Finset Item) (allocation : Allocation Agent Item) : Prop :=
  ¬ ∃ improvement : Allocation Agent Item,
    IsAllocationOf improvement chores ∧
    ParetoDominatesForChores cost allocation improvement
```

Recorded source-to-library screening Verdict: matches

Reason: The source forbids an allocation that weakly decreases every agent's cost and strictly decreases one. The Lean definition states that condition over the complete-allocation convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.AdditiveChoreInstance.IsParetoOptimal

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:280-287

Verbatim paper-source connection

```
\begin{definition}[Pareto-Optimality]
  An allocation  $Y=(Y_1,\ldots,Y_n)$  \emph{Pareto-dominates} an allocation  $X=(X_1,\ldots,X_n)$  if
  \begin{itemize}
    \item  $c_i(Y_i) \leq c_i(X_i)$  holds for all  $i \in \{1,\ldots,n\}$ , and
    \item there exists  $i \in \{1,\ldots,n\}, \;$   $c_i(Y_i) < c_i(X_i)$ .
  \end{itemize}
  An allocation  $X=(X_1,\ldots,X_n)$  is \emph{Pareto-optimal} if it is not Pareto-dominated by any allocation.
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : DecidableEq Item]
  (choreInstance : EconCSLib.FairDivision.AdditiveChoreInstance Agent Item)
  (allocation : EconCSLib.FairDivision.Allocation Agent Item),
  EconCSLib.FairDivision.ParetoOptimalForChores (EconCSLib.FairDivision.additiveChoreCost choreInstance.cost)
  choreInstance.chores allocation
```

Exact Lean library declaration

```
def IsParetoOptimal {Agent Item : Type*} [DecidableEq Item]
  (choreInstance : AdditiveChoreInstance Agent Item)
  (allocation : Allocation Agent Item) : Prop :=
  ParetoOptimalForChores (additiveChoreCost choreInstance.cost) choreInstance.chores allocation
```

Recorded source-to-library screening Verdict: matches

Reason: This specializes the source Pareto-optimality definition to all chores and the instance's additive cost. It is exactly the source notion, with feasibility supplied by the allocation convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.DoesNotStronglyEnvyForChores

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:260-275

Verbatim paper-source connection

```
\begin{definition}[EFX for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{EFX} if for every pair of agents  $i,j$  we have either
\begin{itemize}
\item  $X_i=\emptyset$ , or
\item for every item  $g\in X_i$ ,  $c_i(X_i\setminus\{g\})\leq c_i(X_j)$ .
\end{itemize}
Equivalently, if
\[\tau_i(X):=\tau_i(X_i)=
\begin{cases}
0, & \& X_i=\emptyset, \\
c_i(X_i)-\min_{g\in X_i}c_i(g), & \& X_i\neq\emptyset,
\end{cases}
\]
then  $X$  is EFX if and only if  $\tau_i(X)\leq c_i(X_j)$  for all  $i,j$ .
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
  (allocation : EconCSLib.FairDivision.Allocation Agent Item) (i j : Agent),
  allocation i = ∅ ∨ ∀ item ∈ allocation i, cost i (allocation i \ {item}) ≤ cost i (allocation j)
```

Exact Lean library declaration

```
def DoesNotStronglyEnvyForChores [DecidableEq Item]
  (cost : Agent → Bundle Item → ℝ) (allocation : Allocation Agent Item)
  (i j : Agent) : Prop :=
  allocation i = ∅ ∨
  ∀ item ∈ allocation i, cost i (allocation i \ {item}) ≤ cost i (allocation j)
```

Recorded source-to-library screening Verdict: matches

Reason: The displayed EFX definition gives exactly the ordered-pair alternative used here: an empty own bundle, or the universal one-item-removal cost inequality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EnvyFreeForChores

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:253-255

Verbatim paper-source connection

```
\begin{definition}[EF for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{envy-free} if for every pair of agents  $i,j$  we have  $c_i(X_i)\leq c_i(X_j)$ .
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
  (allocation : EconCSLib.FairDivision.Allocation Agent Item) (i j : Agent),
  cost i (allocation i) ≤ cost i (allocation j)
```

Exact Lean library declaration

```
-- An allocation is envy-free for chores when each agent's own cost is no
greater than their cost for any other agent's bundle. -/
def EnvyFreeForChores (cost : Agent → Bundle Item → ℝ)
  (allocation : Allocation Agent Item) : Prop :=
  ∀ i j, cost i (allocation i) ≤ cost i (allocation j)
```

Recorded source-to-library screening Verdict: matches

Reason: Both the source definition and Lean declaration require every agent's own bundle to cost no more than every comparison bundle under that agent's cost function.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.IsBiValuedChoreCost

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:532-534

Verbatim paper-source connection

```
\begin{theorem}\label{thm:four}
  For four agents, there exists an EFX allocation in every bi-valued instance.
\end{theorem}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p, q, r \geq 0$ such that $c_i(g_j) \in \{p, q, r\}$ for any $i \in N$ and $j \in M$.
 $\hookrightarrow$ N and $j \in M$.
The cost function is bi-valued if there exist constants $p, q \geq 0$ such that $c_i(g_j) \in \{p, q\}$ for any $i \in N$ and $j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost
 $\hookrightarrow$ functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1, r\}$
 $\hookrightarrow$ (for any $i \in N$ and $j \in M$) instead.
We say that the item g_j is ``large'' for agent  $i$  if  $c_i(g_j) = r$  and it is ``small'' for agent i if $c_i(g_j) = 1$.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : EconCSLib.FairDivision.ChoreCost Agent Item),
  ∃ p q, 0 ≤ p ∧ 0 ≤ q ∧ ∀ (agent : Agent) (item : Item), cost agent item = p ∨ cost agent item = q
```

Exact Lean library declaration

```
def IsBiValuedChoreCost (cost : ChoreCost Agent Item) : Prop :=
  ∃ p q : ℝ, 0 ≤ p ∧ 0 ≤ q ∧
    ∀ agent item, cost agent item = p ∨ cost agent item = q
```

Recorded source-to-library screening Verdict: matches

Reason: The source defines bi-valued costs by two nonnegative constants with every pointwise item cost equal to one of them. The Lean predicate is that exact quantified condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.IsSmallChore

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:532-534

Verbatim paper-source connection

```
\begin{theorem}\label{thm:four}
  For four agents, there exists an EFX allocation in every bi-valued instance.
\end{theorem}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p, q, r \geq 0$ such that $c_i(g_j) \in \{p, q, r\}$ for any $i \in N$ and $j \in M$.
 $\hookrightarrow$ N and $j \in M$.
The cost function is bi-valued if there exist constants $p, q \geq 0$ such that $c_i(g_j) \in \{p, q\}$ for any $i \in N$ and $j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost
 $\hookrightarrow$ functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1, r\}$
 $\hookrightarrow$ (for any $i \in N$ and $j \in M$) instead.
We say that the item g_j is ``large'' for agent  $i$  if  $c_i(g_j) = r$  and it is ``small'' for agent i if $c_i(g_j) = 1$.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : EconCSLib.FairDivision.ChoreCost Agent Item) (agent : Agent)
  (item : Item), cost agent item = 1
```

Exact Lean library declaration

```
def IsSmallChore (cost : ChoreCost Agent Item)
  (agent : Agent) (item : Item) : Prop :=
  cost agent item = 1
```

Recorded source-to-library screening Verdict: matches

Reason: The source says a chore is small for an agent exactly when its cost is 1. The Lean predicate is that equality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

ownSmallChoreSet

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:752

Verbatim paper-source connection

An allocation $P=(P_1, P_2, P_3, P_4)$ of $M_{\{0\}}$ is *\emph{canonical}* if, for each agent i , $|P_i| \in \{a, a+1\}$ and P_i contains $\min\{|P_i|, n_i\}$ items that are small for agent i .

[Next verbatim source excerpt]

We will assume $r \geq 2$ in the remaining part of this section.

For a chore g , let

$$S(g) = \{i \in N : c_i(g) = 1\}$$

be the set of agents for whom g is small. We partition the chores into

$$M_{\{0\}} = \{g : |S(g)| \leq 1\}, \text{ } M_2 = \{g : |S(g)| = 2\}, \text{ } M_{\{3,4\}} = \{g : |S(g)| \geq 3\}.$$

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

$$n_i = \left| \{g \in M_{\{0\}} : S(g) = \{i\}\} \right|$$

be the number of $M_{\{0\}}$ items uniquely small for i . Items in $M_{\{0\}}$ with $S(g) = \emptyset$ are all-large items.

Let $|M_{\{0\}}| = 4a + b$ with $b \in \{0, 1, 2, 3\}$.

As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.

We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items.

This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
  {Item : Type u_2} →
    (cost : EconCSLib.FairDivision.ChoreCost Agent Item) →
      (chores : Finset Item) → (agent : Agent) → {item ∈ chores | EconCSLib.FairDivision.IsSmallChore cost agent item}
```

Exact Lean library declaration

```
noncomputable def ownSmallChoreSet (cost : ChoreCost Agent Item)
  (chores : Finset Item) (agent : Agent) : Finset Item := by
  classical
  exact chores.filter fun item => IsSmallChore cost agent item
```

Recorded source-to-library screening Verdict: matches

Reason: The source n_i counts $M_{\{0\}}$ chores uniquely small for agent i . Applied to an allocated bundle, the Lean finite filter is exactly the set whose cardinality the canonical definition uses.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

IsCanonicalSmallChoreAllocation

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:752

Verbatim paper-source connection

An allocation $P=(P_1, P_2, P_3, P_4)$ of $M_{\{0\}}$ is *canonical* if, for each agent i , $|P_i| \in \{a, a+1\}$ and P_i contains $\min\{|P_i|, n_i\}$ items that are small for agent i .

[Next verbatim source excerpt]

We will assume $r \geq 2$ in the remaining part of this section.
For a chore g , let

```
\[
  S(g) = \{i \in N : c_i(g) = 1\}
\]
```

be the set of agents for whom g is small. We partition the chores into

```
\[
M_{\{0\}} = \{g : |S(g)| \leq 1\}, \text{ } \sqcup
M_2 = \{g : |S(g)| = 2\}, \text{ } \sqcup
M_{\{34\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

```
\[
  n_i = |\{g \in M_{\{0\}} : S(g) = \{i\}\}|
\]
```

be the number of $M_{\{0\}}$ items uniquely small for i . Items in $M_{\{0\}}$ with $S(g) = \emptyset$ are all-large items.
Let $|M_{\{0\}}| = 4a + b$ with $b \in \{0, 1, 2, 3\}$.
As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.

We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items. This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (cost : EconCSLib.FairDivision.ChoreCost Agent Item)
  (chores : Finset Item) (quota : Agent → ℕ) (allocation : EconCSLib.FairDivision.Allocation Agent Item),
  EconCSLib.FairDivision.IsAllocationOf allocation chores ∧
  ∀ (agent : Agent),
    Finset.card (allocation agent) = quota agent ∧
    (EconCSLib.FairDivision.ownSmallChoreSet cost (allocation agent) agent).card =
      min (quota agent) (EconCSLib.FairDivision.ownSmallChoreSet cost chores agent).card
```

Exact Lean library declaration

```
noncomputable def IsCanonicalSmallChoreAllocation [DecidableEq Item]
  (cost : ChoreCost Agent Item) (chores : Finset Item) (quota : Agent → ℕ)
  (allocation : Allocation Agent Item) : Prop :=
  IsAllocationOf allocation chores ∧
  ∀ agent,
    (allocation agent).card = quota agent ∧
    (ownSmallChoreSet cost (allocation agent) agent).card =
      min (quota agent) (ownSmallChoreSet cost chores agent).card
```

Recorded source-to-library screening Verdict: matches

Reason: The definition requires a complete allocation, exact quotas, and exactly $\min(\text{quota}, \text{own-small-count})$ own-small chores for every agent, matching the source canonical-allocation definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.IsLargeChore

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:532-534

Verbatim paper-source connection

```
\begin{theorem}\label{thm:four}
  For four agents, there exists an EFX allocation in every bi-valued instance.
\end{theorem}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p, q, r \geq 0$ such that $c_i(g_j) \in \{p, q, r\}$ for any $i \in N$ and $j \in M$.
 $\hookrightarrow$ N and $j \in M$.
The cost function is bi-valued if there exist constants $p, q \geq 0$ such that $c_i(g_j) \in \{p, q\}$ for any $i \in N$ and $j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost
 $\hookrightarrow$ functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1, r\}$
 $\hookrightarrow$ (for any $i \in N$ and $j \in M$) instead.
We say that the item g_j is ``large'' for agent  $i$  if  $c_i(g_j) = r$  and it is ``small'' for agent i if $c_i(g_j) = 1$.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : EconCSLib.FairDivision.ChoreCost Agent Item) (r : ℝ) (agent : Agent)
  (item : Item), cost agent item = r
```

Exact Lean library declaration

```
def IsLargeChore (cost : ChoreCost Agent Item) (r : ℝ)
  (agent : Agent) (item : Item) : Prop :=
  cost agent item = r
```

Recorded source-to-library screening Verdict: matches

Reason: The source says a chore is large for an agent exactly when its cost is r . The Lean predicate is that equality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

IsOneOrRChoreCost

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:532-534

Verbatim paper-source connection

```
\begin{theorem}\label{thm:four}
  For four agents, there exists an EFX allocation in every bi-valued instance.
\end{theorem}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p, q, r \geq 0$ such that $c_i(g_j) \in \{p, q, r\}$ for any $i \in N$ and $j \in M$.
 $\hookrightarrow$ N and $j \in M$.
The cost function is bi-valued if there exist constants $p, q \geq 0$ such that $c_i(g_j) \in \{p, q\}$ for any $i \in N$ and $j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost
 $\hookrightarrow$ functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1, r\}$
 $\hookrightarrow$ (for any $i \in N$ and $j \in M$) instead.
We say that the item g_j is ``large'' for agent  $i$  if  $c_i(g_j) = r$  and it is ``small'' for agent i if $c_i(g_j) = 1$.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : EconCSLib.FairDivision.ChoreCost Agent Item) (r : ℝ) (agent : Agent)
  (item : Item), cost agent item = 1 ∨ cost agent item = r
```

Exact Lean library declaration

```
def IsOneOrRChoreCost (cost : ChoreCost Agent Item) (r : ℝ) : Prop :=
  ∀ agent item, cost agent item = 1 ∨ cost agent item = r
```

Recorded source-to-library screening Verdict: matches

Reason: The source's normalized bi-valued model says every individual chore cost is either 1 or r. The Lean predicate is exactly that pointwise disjunction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.smallAgentSet

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:577-581

Verbatim paper-source connection

```
\begin{lemma}[Insertion of an  $M_{\{34\}}$  item]\label{lem:insertion}
Let  $\$X$  be an EFX allocation of a set of chores  $\$T$ . Let  $\$g \in M_{\{34\}}$  be a
chore that is small for at least three agents. Then there is an EFX
allocation of  $\$T \cup \{g\}$ .
\end{lemma}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $\$p, q, r \geq 0$ such that $\$c_i(g_j) \in \{p, q, r\}$ for any $\$i \in N$ and $\$g_j \in M$.
The cost function is bi-valued if there exist constants $\$p, q \geq 0$ such that $\$c_i(g_j) \in \{p, q\}$ for any $\$i \in N$ and $\$g_j \in M$.
For bi-valued cost functions, we will focus on the case $\$0 < p < q$ (the case $\$0 = p < q$ corresponds to the setting with binary cost functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $\$c_i(g_j) \in \{1, r\}$ (for any $\$i \in N$ and $\$g_j \in M$) instead.
We say that the item $\$g_j$ is ``large'' for agent  $\$i$  if  $\$c_i(g_j) = r$  and it is ``small'' for agent $\$i$ if $\$c_i(g_j) = 1$.

[Next verbatim source excerpt]

We will assume $\$r > 2$ in the remaining part of this section.

For a chore $\$g$, let

```
\[
S(g) = \{i \in N : c_i(g) = 1\}
\]
```

be the set of agents for whom $\$g$ is small. We partition the chores into

```
\[
M_{\{01\}} = \{g : |S(g)| \leq 1\}, \quad \text{\\quad}
M_2 = \{g : |S(g)| = 2\}, \quad \text{\\quad}
M_{\{34\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write $\$(i, j)$ for an item small exactly for agents $\$i$ and $\$j$. Thus the items in M_2 form a multigraph on the vertex set $\$N$, where an edge $\$(i, j)$ is a chore small for precisely agents $\$i$ and $\$j$.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
[inst : Fintype Agent] →
[DecidableEq Agent] →
(cost : EconCSLib.FairDivision.ChoreCost Agent Item) →
(item : Item) → {agent | EconCSLib.FairDivision.IsSmallChore cost agent item}
```

Exact Lean library declaration

```
noncomputable def smallAgentSet [Fintype Agent] [DecidableEq Agent]
(cost : ChoreCost Agent Item) (item : Item) : Finset Agent := by
classical
exact Finset.univ.filter fun agent => IsSmallChore cost agent item
```

Recorded source-to-library screening Verdict: matches

Reason: The source defines $S(g)$ as the set of agents for whom g is small. The Lean finite filter over all agents is exactly that set.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.IsSmallForAtLeastThree

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:577-581

Verbatim paper-source connection

```
\begin{lemma}[Insertion of an  $M_{34}$  item]\label{lem:insertion}
Let  $\$X$  be an EFX allocation of a set of chores  $\$T$ . Let  $\$g \in M_{34}$  be a
chore that is small for at least three agents. Then there is an EFX
allocation of  $\$T \cup \{g\}$ .
\end{lemma}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $\$p, q, r \geq 0$ such that $\$c_i(g_j) \in \{p, q, r\}$ for any $\$i \in N$ and $\$g_j \in M$.
The cost function is bi-valued if there exist constants $\$p, q \geq 0$ such that $\$c_i(g_j) \in \{p, q\}$ for any $\$i \in N$ and $\$g_j \in M$.
For bi-valued cost functions, we will focus on the case $\$0 < p < q$ (the case $\$0 = p < q$ corresponds to the setting with binary cost functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $\$c_i(g_j) \in \{1, r\}$ (for any $\$i \in N$ and $\$g_j \in M$) instead.
We say that the item $\$g_j$ is ``large'' for agent  $\$i$  if  $\$c_i(g_j) = r$  and it is ``small'' for agent $\$i$ if $\$c_i(g_j) = 1$.

[Next verbatim source excerpt]

We will assume $\$r > 2$ in the remaining part of this section.
For a chore $\$g$, let
 $\$S(g) = \{i \in N : c_i(g) = 1\}$
be the set of agents for whom $\$g$ is small. We partition the chores into
 $\$M_{01} = \{g : |S(g)| \leq 1\}$,
 $\$M_2 = \{g : |S(g)| = 2\}$,
 $\$M_{34} = \{g : |S(g)| \geq 3\}$.
For an item in $\$M_2$, we write $\$(i, j)$ for an item small exactly for agents $\$i$ and $\$j$. Thus the items in $\$M_2$ form a multigraph on the vertex set $\$N$, where an edge $\$(i, j)$ is a chore small for precisely agents $\$i$ and $\$j$.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype Agent] [inst_1 : DecidableEq Agent]
(cost : EconCSLib.FairDivision.ChoreCost Agent Item) (item : Item),
3 ≤ (EconCSLib.FairDivision.smallAgentSet cost item).card
```

Exact Lean library declaration

```
def IsSmallForAtLeastThree [Fintype Agent] [DecidableEq Agent]
(cost : ChoreCost Agent Item) (item : Item) : Prop :=
3 ≤ (smallAgentSet cost item).card
```

Recorded source-to-library screening Verdict: matches

Reason: The source defines M_{34} as chores whose small-agent set has cardinality at least three. The Lean predicate is exactly that cardinality condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

IsSmallForAtMostOne

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:752

Verbatim paper-source connection

An allocation $P=(P_1, P_2, P_3, P_4)$ of $M_{\{0\}}$ is *\emph{canonical}* if, for each agent i , $|P_i| \in \{a, a+1\}$ and P_i contains $\min\{|P_i|, n_i\}$ items that are small for agent i .

[Next verbatim source excerpt]

We will assume $r \geq 2$ in the remaining part of this section.
For a chore g , let

```
\[
  S(g) = \{i \in N : c_i(g) = 1\}
\]
```

be the set of agents for whom g is small. We partition the chores into

```
\[
M_{\{0\}} = \{g : |S(g)| \leq 1\}, \text{ \quad}
M_2 = \{g : |S(g)| = 2\}, \text{ \quad}
M_{\{3,4\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

```
\[
  n_i = |\{g \in M_{\{0\}} : S(g) = \{i\}\}|
\]
```

be the number of $M_{\{0\}}$ items uniquely small for i . Items in $M_{\{0\}}$ with $S(g) = \emptyset$ are all-large items.
Let $|M_{\{0\}}| = 4a + b$ with $b \in \{0, 1, 2, 3\}$.
As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.

We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items. This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype Agent] [inst_1 : DecidableEq Agent]
  (cost : EconCSLib.FairDivision.ChoreCost Agent Item) (item : Item),
  (EconCSLib.FairDivision.smallAgentSet cost item).card ≤ 1
```

Exact Lean library declaration

```
def IsSmallForAtMostOne [Fintype Agent] [DecidableEq Agent]
  (cost : ChoreCost Agent Item) (item : Item) : Prop :=
  (smallAgentSet cost item).card ≤ 1
```

Recorded source-to-library screening Verdict: matches

Reason: For an M_{01} chore, the source defines the small-agent set to have cardinality at most one. The Lean predicate states that same finite-cardinality condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.IsSmallForExactlyTwo

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:1045-1047

Verbatim paper-source connection

```
\begin{lemma}[EFX Allocation of  $M_2$ ]\label{lem:M2}
There is an EFX allocation of  $M_2$ .
\end{lemma}
```

[Next verbatim source excerpt]

We will assume $r > 2$ in the remaining part of this section.
For a chore g , let

```
\[
S(g) = \{i \in N : c_i(g) = 1\}
\]
```

be the set of agents for whom g is small. We partition the chores into

```
\[
M_{\{01\}} = \{g : |S(g)| \leq 1\}, \quad \text{\\quad}
M_2 = \{g : |S(g)| = 2\}, \quad \text{\\quad}
M_{\{34\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

```
\subsection{The  $M_2$  Multigraph Algorithm}
\label{subsec:2}
We now give a complete allocation procedure for the items in  $M_2$ .
Each item is small for some agents  $i$  and  $j$  and
large for the other two agents, and we say that the type of this item is  $(i, j)$ , or  $ij$  for short.
In this section, we consider the case where the instance only has  $M_{\{2\}}$  items.
We represent the instance by a
multigraph  $R$  on  $N = \{1, 2, 3, 4\}$ , where an item of type  $(i, j)$  is an edge  $(i, j)$ .
We slightly abuse the notation by letting  $R$  also represent the edge set, i.e.,  $R$  exactly corresponds to items in  $M_2$ .
Let  $x_{ij}$  denote the multiplicity of edge  $(i, j)$  and let  $m_R = |R|$ .

For  $U \subseteq N$ , define
\[\operatorname{inc}_R(U) = \left| \{e \in R : e \cap U \neq \emptyset\} \right|
\]
```

and

```
\[
\Delta_R(U) = \frac{|U| m_R}{4} - \operatorname{inc}_R(U).
\]
```

We call U **deficient** if $\Delta_R(U) > 0$.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype Agent] [inst_1 : DecidableEq Agent]
  (cost : EconCSLib.FairDivision.ChoreCost Agent Item) (item : Item),
  (EconCSLib.FairDivision.smallAgentSet cost item).card = 2
```

Exact Lean library declaration

```
def IsSmallForExactlyTwo [Fintype Agent] [DecidableEq Agent]
  (cost : ChoreCost Agent Item) (item : Item) : Prop :=
  (smallAgentSet cost item).card = 2
```

Recorded source-to-library screening Verdict: matches

Reason: The source defines M_2 as chores whose small-agent set has cardinality exactly two. The Lean predicate is that exact cardinality condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

IsSuperCanonicalSmallChoreAllocation

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:755

Verbatim paper-source connection

For $b > 0$, an allocation $P = (P_1, P_2, P_3, P_4)$ is *super-canonical* if it is canonical and satisfies $c_i(P_i) \leq c_i(P_j) - r$ for every short agent i and every long agent j .

[Next verbatim source excerpt]

We will assume $r > 2$ in the remaining part of this section.
For a chore g , let

```
\[
  S(g) = \{i \in N : c_i(g) = 1\}
\]
be the set of agents for whom  $g$  is small. We partition the chores into
\[
M_{\{0\}} = \{g : |S(g)| \leq 1\}, \text{ } \sqcup
M_2 = \{g : |S(g)| = 2\}, \text{ } \sqcup
M_{\{3,4\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

```
\[
  n_i = |\{g \in M_{\{0\}} : S(g) = \{i\}\}|
\]
be the number of  $M_{\{0\}}$  items uniquely small for  $i$ . Items in  $M_{\{0\}}$ 
with  $S(g) = \emptyset$  are all-large items.
Let  $|M_{\{0\}}| = 4a + b$  with  $b \in \{0, 1, 2, 3\}$ .
As mentioned before, we will make sure each agent receives either  $a$  items or  $a+1$  items.
```

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.

We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items. We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items. This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

[Next verbatim source excerpt]

```
\begin{definition}
  An allocation  $P = (P_1, P_2, P_3, P_4)$  of  $M_{\{0\}}$  is canonical if, for each agent  $i$ ,  $|P_i| \in \{a, a+1\}$  and  $P_i$ 
   $\hookrightarrow$  contains  $\min\{|P_i|, n_i\}$  items that are small for agent  $i$ .
   $\hookrightarrow$  In a canonical allocation with  $b > 0$ , agents receiving  $a$  items are called short agents, and agents receiving  $a+1$ 
   $\hookrightarrow$  items are called long agents.
  For  $b = 0$  and  $a > 0$ , every agent in a canonical is both long and short.
\end{definition}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (r : ℝ)
(cost : EconCSLib.FairDivision.ChoreCost Agent Item) (chores : Finset Item) (shortAgents longAgents : Finset Agent)
(quota : Agent → ℕ) (allocation : EconCSLib.FairDivision.Allocation Agent Item),
EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost chores quota allocation ∧
  ∀ shortAgent ∈ shortAgents,
    ∀ longAgent ∈ longAgents,
      EconCSLib.FairDivision.additiveChoreCost cost shortAgent (allocation shortAgent) ≤
        EconCSLib.FairDivision.additiveChoreCost cost shortAgent (allocation longAgent) - r
```

Exact Lean library declaration

```
noncomputable def IsSuperCanonicalSmallChoreAllocation [DecidableEq Item]
(r : ℝ) (cost : ChoreCost Agent Item) (chores : Finset Item)
(shortAgents longAgents : Finset Agent) (quota : Agent → ℕ)
(allocation : Allocation Agent Item) : Prop :=
IsCanonicalSmallChoreAllocation cost chores quota allocation ∧
  ∀ shortAgent ∈ shortAgents, ∀ longAgent ∈ longAgents,
    additiveChoreCost cost shortAgent (allocation shortAgent) ≤
      additiveChoreCost cost shortAgent (allocation longAgent) - r
```

Recorded source-to-library screening Verdict: matches

Reason: The library definition combines canonicity with the source short-agent versus long-agent cost gap. The paper interface supplies the source's cardinality-based short and long sets.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.IsTriValuedChoreCost

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:304-306

Verbatim paper-source connection

```
\begin{theorem}\label{thm:tri}
  For any  $n \geq 4$ , there exists a tri-valued instance with  $n$  agents where no EFX allocation exists.
\end{theorem}
```

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (cost : EconCSLib.FairDivision.ChoreCost Agent Item),
  ∃ p q r,
    0 ≤ p ∧
    0 ≤ q ∧ 0 ≤ r ∧ ∀ (agent : Agent) (item : Item), cost agent item = p ∨ cost agent item = q ∨ cost agent item = r
```

Exact Lean library declaration

```
def IsTriValuedChoreCost (cost : ChoreCost Agent Item) : Prop :=
  ∃ p q r : ℝ, 0 ≤ p ∧ 0 ≤ q ∧ 0 ≤ r ∧
    ∀ agent item, cost agent item = p ∨ cost agent item = q ∨ cost agent item = r
```

Recorded source-to-library screening Verdict: matches

Reason: The source defines tri-valued costs by three nonnegative constants with every pointwise item cost in their set. The Lean predicate is that exact condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims

Review row 1

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:253-255

Verbatim source input

```
\begin{definition}[EF for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{envy-free} if for every pair of agents  $i,j$  we have  $c_i(X_i)\leq c_i(X_j)$ .
\end{definition}
```

Expanded PaperInterface specification

```
∀ (Agent Item : Type) (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
  (allocation : EconCSLib.FairDivision.Allocation Agent Item),
  EconCSLib.FairDivision.EnvFreeForChores cost allocation ↔
    ∀ (i j : Agent), cost i (allocation i) ≤ cost i (allocation j)
```

Lean proof endpoint (built separately)

HT26EFXChores.envyFreeForChores_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The displayed EF condition is the same all-agent, all-comparison bundle-cost inequality as the transparent Spec.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:260-275

Verbatim source input

```
\begin{definition}[EFX for chores]
An allocation  $X=(X_1,\ldots,X_n)$  is \emph{EFX} if for every pair of agents  $i,j$  we have either
\begin{itemize}
\item  $X_i=\emptyset$ , or
\item for every item  $g\in X_i$ ,  $c_i(X_i\setminus\{g\})\leq c_i(X_j)$ .
\end{itemize}
Equivalently, if
\l[
\tau_i(X):=\tau_i(X_i):=
\begin{cases}
0, & X_i=\emptyset, \\
c_i(X_i)-\min_{g\in X_i}c_i(g), & X_i\neq\emptyset,
\end{cases}
\l]
then  $X$  is EFX if and only if  $\tau_i(X)\leq c_i(X_j)$  for all  $i,j$ .
\end{definition}
```

Expanded PaperInterface specification

```
∀ (Agent Item : Type) (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ)
(allocation : EconCSLib.FairDivision.Allocation Agent Item),
EconCSLib.FairDivision.EFXForChores cost allocation ↔
∀ (i j : Agent), allocation i = ∅ ∨ ∀ item ∈ allocation i, cost i (allocation i \ {item}) ≤ cost i (allocation j)
```

Lean proof endpoint (built separately)

HT26EFXChores.efxForChores_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The source's empty-own-bundle alternative and universal one-item-removal inequality are both present in the transparent Spec.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:280-287

Verbatim source input

```
\begin{definition}[Pareto-Optimality]
  An allocation  $Y=(Y_1,\ldots,Y_n)$   $\emph{Pareto-dominates}$  an allocation  $X=(X_1,\ldots,X_n)$  if
  \begin{itemize}
    \item  $c_i(Y_i) \leq c_i(X_i)$  holds for all  $i \in \{1,\ldots,n\}$ , and
    \item there exists  $i \in \{1,\ldots,n\}, \wedge; c_i(Y_i) < c_i(X_i)$ .
  \end{itemize}
  An allocation  $X=(X_1,\ldots,X_n)$  is  $\emph{Pareto-optimal}$  if it is not Pareto-dominated by any allocation.
\end{definition}
```

Expanded PaperInterface specification

```
∀ (Agent Item : Type) (cost : Agent → EconCSLib.FairDivision.Bundle Item → ℝ) (chores : Finset Item)
  (allocation : EconCSLib.FairDivision.Allocation Agent Item),
  EconCSLib.FairDivision.IsAllocationOf allocation chores →
  (EconCSLib.FairDivision.ParetoOptimalForChores cost chores allocation ↔
   → improvement,
   EconCSLib.FairDivision.IsAllocationOf improvement chores ∧
    (∀ (i : Agent), cost i (improvement i) ≤ cost i (allocation i)) ∧
    ∃ i, cost i (improvement i) < cost i (allocation i))
```

Lean proof endpoint (built separately)

HT26EFXChores.paretoOptimalForChores_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The Spec states exactly the source's no-complete-allocation Pareto-dominates condition, including weak improvement for all agents and strict improvement for one.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:752

Verbatim source input

An allocation $P=(P_1,P_2,P_3,P_4)$ of $M_{\{0\}}$ is *\emph{canonical}* if, for each agent i , $|P_i| \in \{a,a+1\}$ and P_i contains $\min\{|P_i|, n_i\}$ items that are small for agent i .

[Next verbatim source excerpt]

We will assume $r \geq 2$ in the remaining part of this section.

For a chore g , let

```
\[
  S(g) = \{i \in N : c_i(g) = 1\}
\]
```

be the set of agents for whom g is small. We partition the chores into

```
\[
  M_{\{0\}} = \{g : |S(g)| \leq 1\}, \text{ \quad}
  M_2 = \{g : |S(g)| = 2\}, \text{ \quad}
  M_{\{34\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write (i,j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i,j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

```
\[
  n_i = \left| \{g \in M_{\{0\}} : S(g) = \{i\}\} \right|
\]
```

be the number of $M_{\{0\}}$ items uniquely small for i . Items in $M_{\{0\}}$ with $S(g) = \emptyset$ are all-large items.

Let $|M_{\{0\}}| = 4a + b$ with $b \in \{0,1,2,3\}$.

As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.

We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items. This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

Expanded PaperInterface specification

```

∀ (Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item) (a b : ℕ)
  (allocation : EconCSLib.FairDivision.Allocation (Fin 4) Item),
  2 < r →
    EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
      b ≤ 3 →
        chores.card = 4 * a + b →
          (∀ item ∈ chores, EconCSLib.FairDivision.IsSmallForAtMostOne cost item) →
            (∃ quota,
              (∀ (agent : Fin 4), quota agent = a ∨ quota agent = a + 1) ∧
              EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost chores quota allocation) ↔
              EconCSLib.FairDivision.IsAllocationOf allocation chores ∧
              ∀ (agent : Fin 4),
                (Finset.card (allocation agent) = a ∨ Finset.card (allocation agent) = a + 1) ∧
                (EconCSLib.FairDivision.ownSmallChoreSet cost (allocation agent) agent).card =
                  min (Finset.card (allocation agent))
                      (EconCSLib.FairDivision.ownSmallChoreSet cost chores agent).card)
            )

```

Lean proof endpoint (built separately)

HT26EFXChores.canonicalSmallChoreAllocation_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The expanded canonical Spec states the complete four-agent M01 allocation condition directly: adjacent bundle sizes and the exact own-small maximum. The source's separate short/long-label convention and its super-canonical definition have their own Specs and review rows.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:753-754

Verbatim source input

In a canonical allocation with $b > 0$, agents receiving a items are called *short* agents, and agents receiving $a+1$ items are called *long* agents.
For $b = 0$ and $a > 0$, every agent in a canonical is both long and short.

[Next verbatim source excerpt]

We will assume $r \geq 2$ in the remaining part of this section.

For a chore g , let

$S(g) = \{i \in N : c_i(g) = 1\}$

be the set of agents for whom g is small. We partition the chores into

$M_1 = \{g : |S(g)| \leq 1\}$,
 $M_2 = \{g : |S(g)| = 2\}$,
 $M_3 = \{g : |S(g)| \geq 3\}$.

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

$n_i = |\{g \in M_1 : S(g) = \{i\}\}|$

be the number of M_1 items uniquely small for i . Items in M_1 with $S(g) = \emptyset$ are all-large items.

Let $|M_1| = 4a + b$ with $b \in \{0, 1, 2, 3\}$.

As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_1|$.

We consider allocations of M_1 with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items. This freedom enables us to append the allocation of M_2 after the allocation of M_1 .

We begin by defining this type of allocations.

begin{definition}

An allocation $P = (P_1, P_2, P_3, P_4)$ of M_1 is *canonical* if, for each agent i , $|P_i| \in \{a, a+1\}$ and P_i contains $\min\{|P_i|, n_i\}$ items that are small for agent i .

Expanded PaperInterface specification

```
∀ (Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item) (a b : ℕ)
  (allocation : EconCSLib.FairDivision.Allocation (Fin 4) Item),
  2 < r →
    EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
      b ≤ 3 →
        chores.card = 4 * a + b →
          (∀ item ∈ chores, EconCSLib.FairDivision.IsSmallForAtMostOne cost item) →
            (EconCSLib.FairDivision.IsAllocationOf allocation chores ∧
              ∀ (agent : Fin 4),
                (Finset.card (allocation agent) = a ∨ Finset.card (allocation agent) = a + 1) ∧
                (EconCSLib.FairDivision.ownSmallChoreSet cost (allocation agent) agent).card =
                  min (Finset.card (allocation agent))
                    (EconCSLib.FairDivision.ownSmallChoreSet cost chores agent).card) →
              ∃ shortAgents longAgents,
                (0 < b →
                  (∀ (agent : Fin 4), agent ∈ shortAgents ↔ Finset.card (allocation agent) = a) ∧
                  (∀ (agent : Fin 4), agent ∈ longAgents ↔ Finset.card (allocation agent) = a + 1) ∧
                  (b = 0 → 0 < a → shortAgents = Finset.univ ∧ longAgents = Finset.univ))
```

Lean proof endpoint (built separately)

HT26EFXChores.canonicalShortLongLabels_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The expanded Spec gives separate short and long agent sets in a canonical allocation: for b positive they are exactly the a -item and $(a+1)$ -item agents, while for b zero and a positive both sets are all four agents. The raw bundle includes the canonical-allocation definition that fixes the convention's scope.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:755

Verbatim source input

For $b > 0$, an allocation $P = (P_1, P_2, P_3, P_4)$ is *super-canonical* if it is canonical and satisfies $c_i(P_i) \leq c_i(P_j) - r$ for every short agent i and every long agent j .

[Next verbatim source excerpt]

We will assume $r > 2$ in the remaining part of this section.

For a chore g , let

$S(g) = \{i \in N : c_i(g) = 1\}$

be the set of agents for whom g is small. We partition the chores into

$M_{\{0\}} = \{g : |S(g)| \leq 1\}$, $M_{\{1\}} = \{g : |S(g)| = 2\}$, $M_{\{2\}} = \{g : |S(g)| = 3\}$.

For an item in $M_{\{2\}}$, we write (i, j) for an item small exactly for agents i and j . Thus the items in $M_{\{2\}}$ form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let $n_i = |\{g \in M_{\{0\}} : S(g) = \{i\}\}|$ be the number of $M_{\{0\}}$ items uniquely small for i . Items in $M_{\{0\}}$ with $S(g) = \emptyset$ are all-large items. Let $|M_{\{0\}}| = 4a + b$ with $b \in \{0, 1, 2, 3\}$. As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

[Next verbatim source excerpt]

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.

We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her

uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items. This freedom enables us to append the allocation of $M_{\{2\}}$ after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

[Next verbatim source excerpt]

[Next verbatim source excerpt]

[Next verbatim source excerpt]

[Next verbatim source excerpt]

[Next verbatim source excerpt]

[Next verbatim source excerpt]

An allocation $P = (P_1, P_2, P_3, P_4)$ of $M_{\{0\}}$ is *canonical* if, for each agent i , $|P_i| \in \{a, a+1\}$ and P_i contains $\min\{|P_i|, n_i\}$ items that are small for agent i . In a canonical allocation with $b > 0$, agents receiving a items are called *short* agents, and agents receiving $a+1$ items are called *long* agents. For $b = 0$ and $a > 0$, every agent in a canonical is both long and short.

Expanded PaperInterface specification

```
(Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item) (a b : ℕ)
  (allocation : EconCSLib.FairDivision.Allocation (Fin 4) Item),
  2 < r →
    EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
      0 < b →
        b ≤ 3 →
          chores.card = 4 * a + b →
            (∀ item ∈ chores, EconCSLib.FairDivision.IsSmallForAtMostOne cost item) →
              ((∃ quota,
                (∀ (agent : Fin 4), quota agent = a ∨ quota agent = a + 1) ∧
                  EconCSLib.FairDivision.IsSuperCanonicalSmallChoreAllocation r cost chores {agent | quota agent = a}
                    {agent | quota agent = a + 1} quota allocation) ↔
                  EconCSLib.FairDivision.IsAllocationOf allocation chores ∧
                    (∀ (agent : Fin 4),
                      (Finset.card (allocation agent) = a ∨ Finset.card (allocation agent) = a + 1) ∧
                        (EconCSLib.FairDivision.ownSmallChoreSet cost (allocation agent) agent).card =
                          min (Finset.card (allocation agent))
                            (EconCSLib.FairDivision.ownSmallChoreSet cost chores agent).card) ∧
                    (∀ (shortAgent longAgent : Fin 4),
                      Finset.card (allocation shortAgent) = a →
                        Finset.card (allocation longAgent) = a + 1 →
                          EconCSLib.FairDivision.additiveChoreCost cost shortAgent (allocation shortAgent) ≤
                            EconCSLib.FairDivision.additiveChoreCost cost shortAgent (allocation longAgent) - r))
              )
```

Lean proof endpoint (built separately)

HT26EFXChores.superCanonicalSmallChoreAllocation_iff_source_definition

Recorded source-to-Spec screening Verdict: matches

Reason: The expanded Spec separately states the source's b -positive super-canonical definition: canonicity plus the cost gap from every short agent to every long agent. Its raw source bundle includes the preceding definitions that determine $M_{\{0\}}$, canonicity, and short/long labels.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 7

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:304-306

Verbatim source input

```
\begin{theorem}\label{thm:tri}
  For any  $n \geq 4$ , there exists a tri-valued instance with  $n$  agents where no EFX allocation exists.
\end{theorem}
```

Expanded PaperInterface specification

```
∀ (n : ℕ),
  4 ≤ n →
  ∃ Item itemDecidableEq choreInstance,
    EconCSLib.FairDivision.IsTriValuedChoreCost choreInstance.cost ∧
    ∀ (allocation : EconCSLib.FairDivision.Allocation (Fin n) Item),
      choreInstance.IsFeasible allocation → ¬choreInstance.IsEFX allocation
```

Lean proof endpoint (built separately)

HT26EFXChores.tri_valued_nonexistence

Recorded source-to-Spec screening Verdict: matches

Reason: The source theorem's quantification over n at least four and existence of a tri-valued n -agent instance with no EFX allocation are represented directly. The finite-item equality instance is Lean representation support, not an additional paper premise.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:340-342

Verbatim source input

```
\begin{proposition}
  No agent can receive more than one item in  $A$ .
\end{proposition}

[Next verbatim source excerpt]

The instance consists of  $4$  agents and  $13$  items.
The items are partitioned into three groups:
 $A = \{a_1, a_2, a_3\}$ ,  $B = \{b_1, b_2, b_3, b_4, b_5\}$ ,  $C = \{c_1, c_2, c_3, c_4, c_5\}$ .
The items group  $A$  consists of ' $\text{large}$ ' items where each agent values each of them at  $20$ .
For each item in  $B$ , agents  $1$  and  $2$  value it at  $1$  and agents  $3$  and  $4$  value it at  $7$ .
For each item in  $C$ , agents  $1$  and  $2$  value it at  $7$  and agents  $3$  and  $4$  value it at  $1$ .

[Next verbatim source excerpt]

In the remaining part of this section, we will show that no EFX allocation exists in this instance.
Suppose for contradiction an EFX allocation  $X = (X_1, X_2, X_3, X_4)$  exists.
Firstly, we will show that no one can receive more than one large item in  $A$ .

[Next verbatim source excerpt]

\begin{definition}[EFX for chores]
  An allocation  $X = (X_1, \dots, X_n)$  is  $\text{EFX}$  if for every pair of agents  $i, j$  we have either
\begin{itemize}
  \item  $X_i = \emptyset$ , or
  \item for every item  $g$  in  $X_i$ ,  $c_i(X_i \setminus \{g\}) \leq c_i(X_j)$ .
\end{itemize}
\end{definition}
Equivalently, if
\l
  \tau_i(X) := \tau_i(X_i) :=
  \begin{cases}
    0, & \text{if } X_i = \emptyset, \\
    c_i(X_i) - \min_{g \in X_i} c_i(g), & \text{if } X_i \neq \emptyset,
  \end{cases}
\l
then  $X$  is EFX if and only if  $\tau_i(X) \leq c_i(X_j)$  for all  $i, j$ .
\end{definition}
```

Expanded PaperInterface specification

```
∀ (allocation : EconCSLib.FairDivision.Allocation (Fin 4) (Fin 13)),
  EconCSLib.FairDivision.IsAllocationOf allocation Finset.univ →
  EconCSLib.FairDivision.EFXForChores
    (EconCSLib.FairDivision.additiveChoreCost fun agent item =>
      if item < 3 then 20 else if agent < 2 then if item < 8 then 1 else 7 else if item < 8 then 7 else 1)
    allocation →
  ∀ (agent : Fin 4), {item ∈ allocation agent | item < 3}.card ≤ 1
```

Lean proof endpoint (built separately)

HT26EFXChores.tri_four_no_two_large

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle includes the displayed four-agent construction and the EFX contradiction setup. The Spec states that each bundle contains at most one A item under precisely that construction and EFX premise.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:363-365

Verbatim source input

```
\begin{proposition}
  For any agent  $i$ , we have  $c_i(X_1) \geq 20$ .
\end{proposition}
```

[Next verbatim source excerpt]

The instance consists of 4 agents and 13 items.
The items are partitioned into three groups:
 $A = \{a_1, a_2, a_3\}$, $B = \{b_1, b_2, b_3, b_4, b_5\}$, $C = \{c_1, c_2, c_3, c_4, c_5\}$.
The items group A consists of 'large' items where each agent values each of them at \$20.
For each item in B , agents 1 and 2 value it at \$1 and agents 3 and 4 value it at \$7.
For each item in C , agents 1 and 2 value it at \$7 and agents 3 and 4 value it at \$1.

[Next verbatim source excerpt]

In the remaining part of this section, we will show that no EFX allocation exists in this instance.
Suppose for contradiction an EFX allocation $X = (X_1, X_2, X_3, X_4)$ exists.
Firstly, we will show that no one can receive more than one large item in A .

[Next verbatim source excerpt]

With the above proposition, it must be that one agent does not receive any large item in A and each of the remaining three
↪ agents receives exactly one item in A .
Suppose without loss of generality that agent 1 does not receive any item from A .

[Next verbatim source excerpt]

```
\begin{definition}[EFX for chores]
  An allocation  $X = (X_1, \dots, X_n)$  is \emph{EFX} if for every pair of agents  $i, j$  we have either
\begin{itemize}
  \item  $X_i = \emptyset$ , or
  \item for every item  $g \in X_i$ ,  $c_i(X_i \setminus \{g\}) \leq c_i(X_j)$ .
\end{itemize}
  Equivalently, if
\{
  \tau_i(X) := \tau_i(X_i) =
  \begin{cases}
    0, & \text{if } X_i = \emptyset, \\
    c_i(X_i) - \min_{g \in X_i} c_i(g), & \text{if } X_i \neq \emptyset,
  \end{cases}
\}
  then  $X$  is EFX if and only if  $\tau_i(X) \leq c_i(X_j)$  for all  $i, j$ .
\end{definition}
```

Expanded PaperInterface specification

```
∀ (allocation : EconCSLib.FairDivision.Allocation (Fin 4) (Fin 13)),
  EconCSLib.FairDivision.IsAllocationOf allocation Finset.univ →
  EconCSLib.FairDivision.EFXForChores
    (EconCSLib.FairDivision.additiveChoreCost fun agent item =>
      if item < 3 then 20 else if agent < 2 then if item < 8 then 1 else 7 else if item < 8 then 7 else 1)
    allocation →
  {item ∈ allocation 0 | item < 3}.card = 0 →
  (∀ (agent : Fin 4), agent ≠ 0 → {item ∈ allocation agent | item < 3}.card = 1) →
  ∀ (agent : Fin 4),
    20 ≤
      EconCSLib.FairDivision.additiveChoreCost
        (fun agent item =>
          if item < 3 then 20
          else if agent < 2 then if item < 8 then 1 else 7 else if item < 8 then 7 else 1)
        agent (allocation 0))
```

Lean proof endpoint (built separately)

HT26EFXChores.tri_four_no_a_bundle_expensive

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle includes the construction, the EFX setting, and the stated one-no-A/three-one-A distribution after the source's without-loss-of-generality choice. The Spec expresses that chosen case and the resulting cost-at-least-20 conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:416-418

Verbatim source input

```
\begin{theorem}\label{thm:EFXP0}  
  For every  $n \geq 4$  and  $r > \lceil n/2 \rceil + 1$ , there exists a  $(1,r)$ -bi-valued instance with  $n$  agents where every EFX  
   $\hookrightarrow$  allocation is not Pareto-optimal.  
\end{theorem}
```

Expanded PaperInterface specification

```
∀ (n : ℕ),  
  4 ≤ n →  
    ∀ (r : ℝ),  
      ↑((n + 1) / 2) + 1 < r →  
        ∃ Item itemDecidableEq choreInstance,  
          EconCSLib.FairDivision.IsOneOrRChoreCost choreInstance.cost r ∧  
            ∀ (allocation : EconCSLib.FairDivision.Allocation (Fin n) Item),  
              choreInstance.IsFeasible allocation →  
                choreInstance.IsEFX allocation → ¬choreInstance.IsParetoOptimal allocation
```

Lean proof endpoint (built separately)

HT26EFXChores.efx_pareto_incompatibility

Recorded source-to-Spec screening Verdict: matches

Reason: The source theorem's n , r threshold, existence of a $(1,r)$ -bi-valued instance, and universal EFX-implies-not-Pareto-optimal conclusion are represented directly. The Spec no longer strengthens the source theorem by fixing a particular item carrier size.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:461-463

Verbatim source input

```
\begin{proposition}\label{prop:P0-exist-large}
  For every  $X$   $i$ , there exists  $g$  in  $X_i$  with  $c_i(g)=r$ .
\end{proposition}

[Next verbatim source excerpt]

\begin{theorem}\label{thm:EFXP0}
  For every  $n \geq 4$  and  $r > \lceil n/2 \rceil + 1$ , there exists a  $(1,r)$ -bi-valued instance with  $n$  agents where every EFX
   $\hookrightarrow$  allocation is not Pareto-optimal.
\end{theorem}

[Next verbatim source excerpt]

Let  $t_1 = \lfloor n/2 \rfloor$  and  $t_2 = \lceil n/2 \rceil$ .
Agents are partitioned into two groups  $T_1$  and  $T_2$  with  $|T_1|=t_1$  and  $|T_2|=t_2$ .
There are  $m=2n+1$  items which are partitioned into three groups  $A, B$ , and  $C$ .
 $A$  is the set of large items containing  $n-1$  items which have cost  $r$  to every agent.
 $B$  contains  $t_1+1$  items.
Each agent in  $T_1$  values each item in  $B$  at  $1$  and each item in  $C$  at  $r$ .
 $C$  contains  $t_2+1$  items.
Each agent in  $T_2$  values each item in  $B$  at  $r$  and each item in  $C$  at  $1$ .
Recall that  $r$  can be any real number greater than  $t_2+1$ .
```

Expanded PaperInterface specification

```
∀ (n : ℕ),
  4 ≤ n →
    ∀ (r : ℝ),
      ↑((n + 1) / 2) + 1 < r →
        ∀ (allocation : EconCSLib.FairDivision.Allocation (Fin n) (Fin (2 * n + 1))),
          EconCSLib.FairDivision.IsAllocationOf allocation Finset.univ →
            EconCSLib.FairDivision.EFXForChores
              (EconCSLib.FairDivision.additiveChoreCost fun agent item =>
                if ↑item < n - 1 then r
                else
                  if ↑agent < n / 2 then if ↑item < n + n / 2 then 1 else r else if ↑item < n + n / 2 then r else 1)
              allocation →
                ∀ (agent : Fin n),
                  ∃ item ∈ allocation agent,
                    (if ↑item < n - 1 then r
                     else
                       if ↑agent < n / 2 then if ↑item < n + n / 2 then 1 else r
                       else if ↑item < n + n / 2 then r else 1) =
                      r
```

Lean proof endpoint (built separately)

HT26EFXChores.efx_po_every_agent_large

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle gives the Theorem 2 threshold, the two agent groups, all three item classes, their cardinalities, and their $(1,r)$ costs. The transparent Spec uses $\text{Fin } n$ and $\text{Fin } (2n+1)$, identifies the floor/ceiling groups by index, and states exactly that every EFX bundle contains an r -cost item.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:502-504

Verbatim source input

```
\begin{proposition}\label{prop:r+1r+2}
  For every agent  $i$ , we have  $c_i(X_i) \geq r+1$ . Moreover, there exists an agent  $i$  with  $c_i(X_i) \geq r+2$ .
\end{proposition}

[Next verbatim source excerpt]

\begin{theorem}\label{thm:EFXP0}
  For every  $n \geq 4$  and  $r > \lceil n/2 \rceil + 1$ , there exists a  $(1, r)$ -bi-valued instance with  $n$  agents where every EFX
   $\hookrightarrow$  allocation is not Pareto-optimal.
\end{theorem}

[Next verbatim source excerpt]

Let  $t_1 = \lfloor n/2 \rfloor$  and  $t_2 = \lceil n/2 \rceil$ .
Agents are partitioned into two groups  $T_1$  and  $T_2$  with  $|T_1| = t_1$  and  $|T_2| = t_2$ .
There are  $m = 2n + 1$  items which are partitioned into three groups  $A, B$ , and  $C$ .
 $A$  is the set of large items containing  $n - 1$  items which have cost  $r$  to every agent.
 $B$  contains  $t_1 + 1$  items.
Each agent in  $T_1$  values each item in  $B$  at  $1$  and each item in  $C$  at  $r$ .
 $C$  contains  $t_2 + 1$  items.
Each agent in  $T_2$  values each item in  $B$  at  $r$  and each item in  $C$  at  $1$ .
Recall that  $r$  can be any real number greater than  $t_2 + 1$ .
```

Expanded PaperInterface specification

```
∀ (n : ℕ),
  4 ≤ n →
    ∀ (r : ℝ),
      ↑((n + 1) / 2) + 1 < r →
        ∀ (allocation : EconCSLib.FairDivision.Allocation (Fin n) (Fin (2 * n + 1))),
          EconCSLib.FairDivision.IsAllocationOf allocation Finset.univ →
            EconCSLib.FairDivision.EFXForChores
              (EconCSLib.FairDivision.additiveChoreCost fun agent item =>
                if ↑item < n - 1 then r
                else
                  if ↑agent < n / 2 then if ↑item < n + n / 2 then 1 else r else if ↑item < n + n / 2 then r else 1)
              allocation →
                (∀ (agent : Fin n),
                  r + 1 ≤
                    EconCSLib.FairDivision.additiveChoreCost
                      (fun agent item =>
                        if ↑item < n - 1 then r
                        else
                          if ↑agent < n / 2 then if ↑item < n + n / 2 then 1 else r
                          else if ↑item < n + n / 2 then r else 1)
                      agent (allocation agent)) ∧
                  ∃ agent,
                    r + 2 ≤
                      EconCSLib.FairDivision.additiveChoreCost
                        (fun agent item =>
                          if ↑item < n - 1 then r
                          else
                            if ↑agent < n / 2 then if ↑item < n + n / 2 then 1 else r
                            else if ↑item < n + n / 2 then r else 1)
                        agent (allocation agent))
```

Lean proof endpoint (built separately)

HT26EFXChores.efx_po_cost_lower_bounds

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle gives the same Theorem 2 construction and the proposition's two lower bounds. The transparent Spec has the same n and r threshold, the same finite cost table, EFX feasibility premise, every-agent $r+1$ bound, and one-agent $r+2$ bound.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:532-534

Verbatim source input

```
\begin{theorem}\label{thm:four}
  For four agents, there exists an EFX allocation in every bi-valued instance.
\end{theorem}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p, q, r \geq 0$ such that $c_i(g_j) \in \{p, q, r\}$ for any $i \in N$ and $j \in M$.
The cost function is bi-valued if there exist constants $p, q \geq 0$ such that $c_i(g_j) \in \{p, q\}$ for any $i \in N$ and $j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1, r\}$ (for any $i \in N$ and $j \in M$) instead.
We say that the item g_j is ``large'' for agent  $i$  if  $c_i(g_j) = r$  and it is ``small'' for agent i if $c_i(g_j) = 1$.

Expanded PaperInterface specification

```
∀ (Item : Type) (choreInstance : EconCSLib.FairDivision.AdditiveChoreInstance (Fin 4) Item),
  EconCSLib.FairDivision.IsBiValuedChoreCost choreInstance.cost →
    ∃ allocation, choreInstance.IsFeasible allocation ∧ choreInstance.IsEFX allocation
```

Lean proof endpoint (built separately)

HT26EFXChores.four_agent_bi_valued_efx_exists

Recorded source-to-Spec screening Verdict: matches

Reason: The source theorem asserts EFX existence for every four-agent bi-valued instance. Its pinned definition gives two nonnegative cost values; the transparent Spec quantifies over exactly a four-agent finite additive-chore instance satisfying that bi-valued condition and concludes a feasible EFX allocation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 14

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:577-581

Verbatim source input

```
\begin{lemma}[Insertion of an  $M_{\{34\}}$  item]\label{lem:insertion}
Let  $X$  be an EFX allocation of a set of chores  $\mathcal{T}$ . Let  $g \in M_{\{34\}}$  be a
chore that is small for at least three agents. Then there is an EFX
allocation of  $\mathcal{T} \cup \{g\}$ .
\end{lemma}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p, q, r \geq 0$ such that $c_i(g_j) \in \{p, q, r\}$ for any $i \in N$ and $j \in M$.
The cost function is bi-valued if there exist constants $p, q \geq 0$ such that $c_i(g_j) \in \{p, q\}$ for any $i \in N$ and $j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1, r\}$ for any $i \in N$ and $j \in M$ instead.
We say that the item g_j is "large" for agent i if $c_i(g_j) = r$ and it is "small" for agent i if $c_i(g_j) = 1$.

[Next verbatim source excerpt]

We will assume $r \geq 2$ in the remaining part of this section.

For a chore g , let

$$S(g) = \{i \in N : c_i(g) = 1\}$$

be the set of agents for whom g is small. We partition the chores into

$$M_1 = \{g : |S(g)| \leq 1\}, \quad M_2 = \{g : |S(g)| = 2\}, \quad M_3 = \{g : |S(g)| \geq 3\}.$$

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

Expanded PaperInterface specification

```

  ∀ (Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item) (item : Item),
  2 < r →
    EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
      item ∉ chores →
        EconCSLib.FairDivision.IsSmallForAtLeastThree cost item →
          ∀ (allocation : EconCSLib.FairDivision.Allocation (Fin 4) Item),
            EconCSLib.FairDivision.IsAllocationOf allocation chores →
              EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) allocation →
                ∃ extended,
                  EconCSLib.FairDivision.IsAllocationOf extended (insert item chores) ∧
                    EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) extended

```

Lean proof endpoint (built separately)

HT26EFXChores.m34_insertion

Recorded source-to-Spec screening Verdict: matches

Reason: The raw lemma and pinned normalization/M34 definitions state an EFX allocation of T , a new item outside T that is small for at least three agents, and an EFX allocation after insertion. The transparent Spec expands those same 1, r , $r > 2$, feasibility, EFX, freshness, and small-agent conditions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 15

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:706-724

Verbatim source input

```
\begin{lemma}[Composition]\label{lem:composition}
Let  $P=(P_1,\ldots,P_n)$  be an allocation of some items and  $B=(B_1,\ldots,B_n)$  an allocation of disjoint
items. If
\[\begin{aligned} & c_i(P_i) \leq c_i(P_j) \quad \text{for all } i,j \\ & \text{and} \\ & c_i(B_i) - 1 \leq c_i(B_j) \quad \text{for all } i,j, \end{aligned}\]
```

then the combined allocation $X_i = P_i \cup B_i$ is EFX. %provided every agent whose second inequality is tight has a small item in
→ her final bundle.
More generally, X is EFX whenever

```
\[\begin{aligned} & c_i(P_i) - c_i(P_j) \\ & \leq c_i(B_j) - c_i(B_i) + \min_{g \in X_i} c_i(g) \end{aligned}\]
```

→ \quad \text{for all } i,j.

If the inequalities above hold only for a particular pair of agents (i,j) , then i does not strongly envy j .

```
\end{lemma}
```

[Next verbatim source excerpt]

We say that the cost function is tri-valued if there exist constants $p,q,r \geq 0$ such that $c_i(g_j) \in \{p,q,r\}$ for any $i \in N$
→ N and $g_j \in M$.
The cost function is bi-valued if there exist constants $p,q \geq 0$ such that $c_i(g_j) \in \{p,q\}$ for any $i \in N$ and $g_j \in M$.
For bi-valued cost functions, we will focus on the case $0 < p < q$ (the case $0 = p < q$ corresponds to the setting with binary cost
→ functions and we know an allocation that is EFX and Pareto-optimal always exists), and we will assume $c_i(g_j) \in \{1,r\}$
→ (for any $i \in N$ and $g_j \in M$) instead.
We say that the item g_j is ``large'' for agent  $i$  if  $c_i(g_j) = r$  and it is ``small'' for agent i if $c_i(g_j) = 1$.

Expanded PaperInterface specification

```
∀ (Agent Item : Type) (cost : EconCSLib.FairDivision.ChoreCost Agent Item) (leftChores rightChores : Finset Item)
(leftAllocation rightAllocation : EconCSLib.FairDivision.Allocation Agent Item),
Disjoint leftChores rightChores →
EconCSLib.FairDivision.IsAllocationOf leftAllocation leftChores →
EconCSLib.FairDivision.IsAllocationOf rightAllocation rightChores →
(∀ (agent : Agent) (item : Item), 1 ≤ cost agent item) →
EconCSLib.FairDivision.IsAllocationOf (fun agent => leftAllocation agent ∪ rightAllocation agent)
(leftChores ∪ rightChores) ∧
((EconCSLib.FairDivision.EnvyFreeForChores (EconCSLib.FairDivision.additiveChoreCost cost) leftAllocation ∧
  ∀ (i j : Agent),
    EconCSLib.FairDivision.additiveChoreCost cost i (rightAllocation i) - 1 ≤
    EconCSLib.FairDivision.additiveChoreCost cost i (rightAllocation j)) →
  EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) fun agent =>
    leftAllocation agent ∪ rightAllocation agent) ∧
  ((∀ (i j : Agent) (hnonempty : Finset.Nonempty (leftAllocation i ∪ rightAllocation i)),
    EconCSLib.FairDivision.additiveChoreCost cost i (leftAllocation i) -
    EconCSLib.FairDivision.additiveChoreCost cost i (leftAllocation j) ≤
    EconCSLib.FairDivision.additiveChoreCost cost i (rightAllocation j) -
    EconCSLib.FairDivision.additiveChoreCost cost i (rightAllocation i) +
    (Finset.image (cost i) (leftAllocation i ∪ rightAllocation i)).min' ... →
    EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) fun agent =>
      leftAllocation agent ∪ rightAllocation agent) ∧
    ∀ (i j : Agent) (hnonempty : Finset.Nonempty (leftAllocation i ∪ rightAllocation i)),
    EconCSLib.FairDivision.additiveChoreCost cost i (leftAllocation i) -
    EconCSLib.FairDivision.additiveChoreCost cost i (leftAllocation j) ≤
    EconCSLib.FairDivision.additiveChoreCost cost i (rightAllocation j) -
    EconCSLib.FairDivision.additiveChoreCost cost i (rightAllocation i) +
    (Finset.image (cost i) (leftAllocation i ∪ rightAllocation i)).min' ... →
    EconCSLib.FairDivision.DoesNotStronglyEnvyForChores (EconCSLib.FairDivision.additiveChoreCost cost)
      (fun agent => leftAllocation agent ∪ rightAllocation agent) i j
```

Lean proof endpoint (built separately)

HT26EFXChores.composition

Recorded source-to-Spec screening Verdict: matches

Reason: The displayed source lemma states disjoint allocations, the first and second cost-gap alternatives, the combined EFX conclusion, and the single-pair no-strong-envy conclusion. The transparent Spec gives those alternatives and conclusions explicitly, with the pinned normalized nonnegative-cost convention supplying the visible lower-bound fact used in the first alternative.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 16

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:760-767

Verbatim source input

```
\begin{lemma}\label{lem:canonical}
  We have the following properties regarding canonical allocations.
  \begin{enumerate}[label=(\alph*)]
    \item Every canonical allocation is EFX. For  $b=0$ , every canonical allocation is envy-free.
    \item For any  $b>0$ , there always exists a super-canonical allocation.
    \item If  $a>0$  and  $b=2$ , for any partition  $N_1, N_2$  of  $N$  with  $|N_1|=|N_2|=2$ , there exist  $i \in N_1$  and  $j \in N_2$ 
       $\rightarrow$  such that there exists a super-canonical allocation  $P$  with  $|P_i|=|P_j|=a$ .
  \end{enumerate}
\end{lemma}
```

[Next verbatim source excerpt]

We will assume $r>2$ in the remaining part of this section.

For a chore g , let

```
\[
  S(g) = \{i \in N : c_i(g) = 1\}
\]
```

be the set of agents for whom g is small. We partition the chores into

```
\[
  M_{\{01\}} = \{g : |S(g)| \leq 1\}, \quad \text{\\quad}
  M_2 = \{g : |S(g)| = 2\}, \quad \text{\\quad}
  M_{\{34\}} = \{g : |S(g)| \geq 3\}.
\]
```

For an item in M_2 , we write (i, j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i, j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

For each agent i , let

```
\[
  n_i = \left| \{g \in M_{\{01\}} : S(g) = \{i\}\} \right|
\]
```

be the number of $M_{\{01\}}$ items uniquely small for i . Items in $M_{\{01\}}$ with $S(g) = \emptyset$ are all-large items.

Let $|M_{\{01\}}| = 4a + b$ with $b \in \{0, 1, 2, 3\}$.

As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{01\}}|$.

We consider allocations of $M_{\{01\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her

$\rightarrow$ uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.

We will show that all such allocations satisfy EFX.

It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items.

This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{01\}}$.

We begin by defining this type of allocations.

Expanded PaperInterface specification

```

 $\forall$  (Item : Type) (r :  $\mathbb{R}$ ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item) (a b :  $\mathbb{N}$ ),
  2 < r  $\rightarrow$ 
    EconCSLib.FairDivision.IsOneOrRChoreCost cost  $\rightarrow$ 
      b  $\leq$  3  $\rightarrow$ 
        chores.card = 4 * a + b  $\rightarrow$ 
          ( $\forall$  item  $\in$  chores, EconCSLib.FairDivision.IsSmallForAtMostOne cost item)  $\rightarrow$ 
            ( $\forall$  (quota : Fin 4  $\rightarrow$   $\mathbb{N}$ ) (allocation : EconCSLib.FairDivision.Allocation (Fin 4) Item),
              ( $\forall$  (agent : Fin 4), quota agent = a  $\vee$  quota agent = a + 1)  $\rightarrow$ 
                EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost chores quota allocation  $\rightarrow$ 
                  EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) allocation)  $\wedge$ 
            (b = 0  $\rightarrow$ 
              ( $\forall$  (quota : Fin 4  $\rightarrow$   $\mathbb{N}$ ) (allocation : EconCSLib.FairDivision.Allocation (Fin 4) Item),
                ( $\forall$  (agent : Fin 4), quota agent = a)  $\rightarrow$ 
                  EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost chores quota allocation  $\rightarrow$ 
                    EconCSLib.FairDivision.EnvyFreeForChores (EconCSLib.FairDivision.additiveChoreCost cost)
                      allocation)  $\wedge$ 
              (0 < b  $\rightarrow$ 
                 $\exists$  quota allocation,
                  ( $\forall$  (agent : Fin 4), quota agent = a  $\vee$  quota agent = a + 1)  $\wedge$ 
                    EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost chores quota allocation  $\wedge$ 
                     $\forall$  (i j : Fin 4),
                      quota i = a  $\rightarrow$ 
                        quota j = a + 1  $\rightarrow$ 
                          EconCSLib.FairDivision.additiveChoreCost cost i (allocation i)  $\leq$ 
                            EconCSLib.FairDivision.additiveChoreCost cost i (allocation j) - r)  $\wedge$ 
              (0 < a  $\rightarrow$ 
                b = 2  $\rightarrow$ 
                   $\forall$  (N1 N2 : Finset (Fin 4)),
                    N1  $\cup$  N2 = Finset.univ  $\rightarrow$ 
                      Disjoint N1 N2  $\rightarrow$ 
                        N1.card = 2  $\rightarrow$ 
                          N2.card = 2  $\rightarrow$ 
                             $\exists$  i  $\in$  N1,
                               $\exists$  j  $\in$  N2,
                                 $\exists$  quota allocation,
                                  ( $\forall$  (agent : Fin 4), quota agent = a  $\vee$  quota agent = a + 1)  $\wedge$ 
                                    EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost chores quota
                                      allocation)  $\wedge$ 

```

```

(∀ (x y : Fin 4),
  quota x = a →
  quota y = a + 1 →
  EconCSLib.FairDivision.additiveChoreCost cost x (allocation x) ≤
  EconCSLib.FairDivision.additiveChoreCost cost x (allocation y) -
  r) ∧
quota i = a ∧ quota j = a)

```

Lean proof endpoint (built separately)

HT26EFXChores.canonical_allocation_properties

Recorded source-to-Spec screening Verdict: matches

Reason: The raw bundle includes the M01 and canonical-allocation definitions as well as the three displayed lemma clauses. The transparent Spec spells out their four-agent, adjacent-quota, canonicity, EFX/envy-free, super-canonical-existence, and selected-pair conclusions without changing a clause.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 17

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:1019-1031

Verbatim source input

```
\begin{lemma}[Balanced orientation]\label{lem:balancedorientation}
Let  $G=(V,E)$  be a multigraph and let  $(b_i)_{i \in V}$  be nonnegative
integers satisfying  $\sum_{i \in V} b_i = |E|$ . There exists an assignment of
every edge to one of its endpoints such that vertex  $i$  receives exactly
 $b_i$  edges if and only if
\begin{equation}\label{eqn:orientation}
\sum_{i \in U} b_i \leq \operatorname{inc}_G(U)
\quad \text{for every } U \subseteq V.
\end{equation}
```

If we have $(\sum_{i \in V} b_i \leq |E|)$ instead, and [\eqref{eqn:orientation}](#) is satisfied, then one can assign some edges to their endpoints so that vertex i receives exactly b_i edges, leaving the remaining edges unassigned.

Expanded PaperInterface specification

```
∀ (Vertex Edge : Type) [inst : Fintype Vertex] (edges : Finset Edge) (endpoints : Edge → Finset Vertex)
  (quota : Vertex → ℕ),
  (∀ edge ∈ edges, (endpoints edge).card = 2) →
  (Finset.univ.sum quota = edges.card →
    ((∃ owner,
      (∀ edge ∈ edges, ∃ vertex, owner edge = some vertex ∧ vertex ∈ endpoints edge) ∧
      ∀ (vertex : Vertex), {edge ∈ edges | owner edge = some vertex}.card = quota vertex) ↔
      ∀ (vertices : Finset Vertex),
        vertices.sum quota ≤ {edge ∈ edges | (endpoints edge ∩ vertices).Nonempty}.card)) ∧
    (Finset.univ.sum quota ≤ edges.card →
      (∀ (vertices : Finset Vertex),
        vertices.sum quota ≤ {edge ∈ edges | (endpoints edge ∩ vertices).Nonempty}.card) →
        ∃ owner,
          (∀ edge ∈ edges, ∀ (vertex : Vertex), owner edge = some vertex → vertex ∈ endpoints edge) ∧
          ∀ (vertex : Vertex), {edge ∈ edges | owner edge = some vertex}.card = quota vertex))
```

Lean proof endpoint (built separately)

HT26EFXChores.balanced_orientation

Recorded source-to-Spec screening Verdict: matches

Reason: The full displayed lemma is sufficient: the Spec gives the exact all-edges endpoint-assignment equivalence at equality and the partial Option-valued assignment conclusion when total demand is at most the number of edges.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 18

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:1045-1047

Verbatim source input

```
\begin{lemma}[EFX Allocation of  $M_2$ ]\label{lem:M2}
There is an EFX allocation of  $M_2$ .
\end{lemma}
```

[Next verbatim source excerpt]

We will assume $r > 2$ in the remaining part of this section.
For a chore g , let
 $S(g) = \{i \in N : c_i(g) = 1\}$
be the set of agents for whom g is small. We partition the chores into
 $M_{\{0\}} = \{g : |S(g)| \leq 1\}$, $M_2 = \{g : |S(g)| = 2\}$, $M_{\{3,4\}} = \{g : |S(g)| \geq 3\}$.
For an item in M_2 , we write (i,j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i,j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

```
\subsection{The  $M_2$  Multigraph Algorithm}
\label{subsec:2}
We now give a complete allocation procedure for the items in  $M_2$ .
Each item is small for some agents  $i$  and  $j$  and
large for the other two agents, and we say that the type of this item is  $(i,j)$ , or  $ij$  for short.
In this section, we consider the case where the instance only has  $M_2$  items.
We represent the instance by a
multigraph  $R$  on  $N = \{1,2,3,4\}$ , where an item of type  $(i,j)$  is an edge  $(i,j)$ .
We slightly abuse the notation by letting  $R$  also represent the edge set, i.e.,  $R$  exactly corresponds to items in  $M_2$ .
Let  $x_{ij}$  denote the multiplicity of edge  $(i,j)$  and let  $m_R = |R|$ .
```

```
For  $U \subseteq N$ , define
 $\operatorname{inc}_R(U) = \left| \{e \in R : e \cap U \neq \emptyset\} \right|$ 
and
 $\Delta_R(U) = \frac{|U| m_R}{4} - \operatorname{inc}_R(U)$ .
We call  $U$  deficient if  $\Delta_R(U) > 0$ .
```

Expanded PaperInterface specification

```
∀ (Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item),
  2 < r →
    EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
      (∀ item ∈ chores, EconCSLib.FairDivision.IsSmallForExactlyTwo cost item) →
        ∃ allocation,
          EconCSLib.FairDivision.IsAllocationOf allocation chores ∧
            EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) allocation
```

Lean proof endpoint (built separately)

HT26EFXChores.m2_efx_allocation

Recorded source-to-Spec screening Verdict: matches

Reason: The raw lemma asserts an EFX allocation of M_2 , and its pinned definition identifies M_2 as items small for exactly two agents in the normalized four-agent model. The transparent Spec has exactly those hypotheses and a feasible EFX allocation conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 19

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:1419-1462

Verbatim source input

```
\begin{lemma}[Properties of the EFX allocations in Lemma~\ref{lem:M2}]{\label{lem:M2properties}}
The EFX allocation  $B=(B_1,\dots,B_4)$  ensured by Lemma~\ref{lem:M2} can be chosen to satisfy the
following additional properties.
\begin{enumerate}[label=(\roman*)]
\item Except in the two configurations described in~(ii), every possible envy is certified by removing a small item. More
 $\hookrightarrow$  precisely,
whenever  $B_i$  contains an item that is small for agent  $i$ ,
\[\bigcup_{i \in N} (B_i - \{c_i(B_i)\}) \cap B_j = \emptyset \quad \text{for all } j \in N,\]
whereas every agent whose bundle contains only large items is envy-free:
\[\bigcup_{i \in N} (B_i - \{c_i(B_i)\}) \cap B_j = \emptyset \quad \text{for all } j \in N.\]
\item Up to relabeling, the only configurations in which an agent whose
bundle contains only large items may fail to be envy-free, and hence
must use a large item as the EFX-removal item, are the following.
\begin{enumerate}[label=(\alph*)]
\item All items have type  $(3,4)$  and
\[\lvert R \rvert = 4q + 3.\]
Then agents  $1,2$  receive  $q$  and  $q+1$  items, respectively, in either
order, while agents  $3,4$  receive  $q+1$  items each. We are free to choose
which of agents  $1,2$  receives  $q+1$  items, and this agent is the unique agent who use a large item as the EFX-removal item.
\item The multiset  $R$  consists of one item of type  $(1,2)$  and
 $4q+3$  items of type  $(3,4)$ . One of agents  $1,2$  receives the unique
type- $(1,2)$  item together with  $q$  type- $(3,4)$  items, the other receives
 $q+1$  type- $(3,4)$  items, and agents  $3,4$  receive  $q+1$  type- $(3,4)$ 
items each. We are free to choose which of agents  $1,2$  receives the unique
type- $(1,2)$  item. The other agent is the unique agent who must use a large
item as the EFX-removal item.
\end{enumerate}
\item If every edge of  $R$  has type  $(1,2)$  or  $(3,4)$ , with
 $x_{\{3,4\}} \geq x_{\{1,2\}} \geq 0$ , then the allocation can be chosen so that agent  $1$ 
is envy-free:
\[\bigcup_{i \in N} (B_i - \{c_i(B_i)\}) \cap B_j = \emptyset \quad \text{for all } j \in N.\]
The same statement holds with agent  $2$  in place of agent  $1$ .
\end{enumerate}
\end{lemma}
```

[Next verbatim source excerpt]

We will assume $r > 2$ in the remaining part of this section.
For a chore g , let

$$S(g) = \{i \in N : c_i(g) = 1\}$$

be the set of agents for whom g is small. We partition the chores into

$$M_1 = \{g : |S(g)| \leq 1\},$$
$$M_2 = \{g : |S(g)| = 2\},$$
$$M_3 = \{g : |S(g)| \geq 3\}.$$

For an item in M_2 , we write (i,j) for an item small exactly for agents i and j . Thus the items in M_2 form a multigraph on the vertex set N , where an edge (i,j) is a chore small for precisely agents i and j .

[Next verbatim source excerpt]

The M_2 Multigraph Algorithm

We now give a complete allocation procedure for the items in M_2 . Each item is small for some agents i and j and large for the other two agents, and we say that the type of this item is (i,j) , or ij for short. In this section, we consider the case where the instance only has M_2 items. We represent the instance by a multigraph R on $N = \{1,2,3,4\}$, where an item of type (i,j) is an edge (i,j) . We slightly abuse the notation by letting R also represent the edge set, i.e., R exactly corresponds to items in M_2 . Let x_{ij} denote the multiplicity of edge (i,j) and let $m_R = |R|$.

For $U \subseteq N$, define

$$\operatorname{inc}_R(U) = \left| \{e \in R : e \cap U \neq \emptyset\} \right|$$

and

$$\Delta_R(U) = \frac{|U| m_R}{4} - \operatorname{inc}_R(U).$$

We call U *deficient* if $\Delta_R(U) > 0$.

Expanded PaperInterface specification

```

∀ (Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item) (chores : Finset Item),
2 < r →
EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
(∀ item ∈ chores, EconCSLib.FairDivision.IsSmallForExactlyTwo cost item) →
∃ allocation,
EconCSLib.FairDivision.IsAllocationOf allocation chores ∧
EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost) allocation ∧
((¬∃ dominant auxiliary q,
dominant.card = 2 ∧
auxiliary.card = 2 ∧
Disjoint dominant auxiliary ∧
((∀ item ∈ chores, EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
chores.card = 4 * q + 3 ∧
∃ exceptionalItem ∈ chores,
EconCSLib.FairDivision.smallAgentSet cost exceptionalItem = auxiliary ∧
(∀ item ∈ chores.erase exceptionalItem,
EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
(chores.erase exceptionalItem).card = 4 * q + 3)) →
(∀ (i : Fin 4),
(∃ item ∈ allocation i, EconCSLib.FairDivision.IsSmallChore cost i item) →
∀ (j : Fin 4),
EconCSLib.FairDivision.additiveChoreCost cost i (allocation i) - 1 ≤
EconCSLib.FairDivision.additiveChoreCost cost i (allocation j)) ∧
∀ (i : Fin 4),
(∀ item ∈ allocation i, EconCSLib.FairDivision.IsLargeChore cost r i item) →
∀ (j : Fin 4),
EconCSLib.FairDivision.additiveChoreCost cost i (allocation i) ≤
EconCSLib.FairDivision.additiveChoreCost cost i (allocation j)) ∧
(∀ (firstType secondType : Finset (Fin 4)),
firstType.card = 2 →
secondType.card = 2 →
Disjoint firstType secondType →
(∀ item ∈ chores,
EconCSLib.FairDivision.smallAgentSet cost item = firstType ∧
EconCSLib.FairDivision.smallAgentSet cost item = secondType) →
{item ∈ chores | EconCSLib.FairDivision.smallAgentSet cost item = secondType}.card ≥
{item ∈ chores | EconCSLib.FairDivision.smallAgentSet cost item = firstType}.card →
∀ agent ∈ firstType,
∃ preferredAllocation,
EconCSLib.FairDivision.IsAllocationOf preferredAllocation chores ∧
EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost)
preferredAllocation ∧
∀ (other : Fin 4),
EconCSLib.FairDivision.additiveChoreCost cost agent
(preferredAllocation agent) ≤
EconCSLib.FairDivision.additiveChoreCost cost agent
(preferredAllocation other)) ∧
∀ (dominant auxiliary : Finset (Fin 4)) (q : ℕ),
dominant.card = 2 →
auxiliary.card = 2 →
Disjoint dominant auxiliary →
∀ special ∈ auxiliary,
∀ companion ∈ auxiliary,
special ≠ companion →
((∀ item ∈ chores, EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
chores.card = 4 * q + 3 ∧
∃ exceptionalItem ∈ chores,
EconCSLib.FairDivision.smallAgentSet cost exceptionalItem = auxiliary ∧
(∀ item ∈ chores.erase exceptionalItem,
EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
(chores.erase exceptionalItem).card = 4 * q + 3) →
∃ exceptionalAllocation,
EconCSLib.FairDivision.IsAllocationOf exceptionalAllocation chores ∧
EconCSLib.FairDivision.EFXForChores
(EconCSLib.FairDivision.additiveChoreCost cost) exceptionalAllocation ∧
(∀ (agent : Fin 4),
agent ≠ special →
∀ (other : Fin 4),
EconCSLib.FairDivision.additiveChoreCost cost agent
(exceptionalAllocation agent) -
1 ≤
EconCSLib.FairDivision.additiveChoreCost cost agent
(exceptionalAllocation other)) ∧
(∀ item ∈ exceptionalAllocation special,
EconCSLib.FairDivision.IsLargeChore cost r special item) ∧
(∀ (other : Fin 4),
other ≠ companion →
EconCSLib.FairDivision.additiveChoreCost cost special
(exceptionalAllocation special) ≤
EconCSLib.FairDivision.additiveChoreCost cost special
(exceptionalAllocation other)) ∧
EconCSLib.FairDivision.additiveChoreCost cost special
(exceptionalAllocation special) -
r ≤
EconCSLib.FairDivision.additiveChoreCost cost special
(exceptionalAllocation companion) ∧
(((∀ item ∈ chores,
EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
chores.card = 4 * q + 3 ∧
∀ (agent : Fin 4),
Finset.card (exceptionalAllocation agent) =
if agent = companion then q else q + 1) ∧
∃ exceptionalItem ∈ chores,
EconCSLib.FairDivision.smallAgentSet cost exceptionalItem =
auxiliary ∧
(∀ item ∈ chores.erase exceptionalItem,

```

```
EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧  
(chores.erase exceptionalItem).card = 4 * q + 3 ∧  
  exceptionalItem ∈ exceptionalAllocation companion ∧  
    ∀ (agent : Fin 4),  
      Finset.card (exceptionalAllocation agent) = q + 1)
```

Lean proof endpoint (built separately)

HT26EFXChores.m2_efx_allocation_properties

Recorded source-to-Spec screening Verdict: matches

Reason: The raw long lemma and pinned M2/type definitions enumerate the ordinary envy certificates, exceptional configurations, preferred-agent branch, and exceptional allocation branch. The transparent Spec expands each of those alternatives on the same normalized four-agent M2 domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 20

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:1709-1712

Verbatim source input

```
\begin{lemma}\label{lem:easycombine}
  Suppose  $P=(P_1,P_2,P_3,P_4)$  is a canonical allocation of  $M_{\{0\}}$ . Suppose some items of  $(M_2)$  are used for gap-filling,
   $\hookrightarrow$  all assigned to short agents, such that  $(P)$  is extended to an envy-free allocation.
  If the residual of  $M_2$  is of an exceptional configuration with exceptional agents  $i$  and  $j$  and both agents are long
   $\hookrightarrow$  agents in the canonical allocation, then there exists an EFX allocation for  $M_{\{0\}} \cup M_2$ .
\end{lemma}
```

[Next verbatim source excerpt]

For each agent i , let

```
\[
  n_i = \left| \{g \in M_{\{0\}} : S(g) = \{i\}\} \right|
\]
```

be the number of $M_{\{0\}}$ items uniquely small for i . Items in $M_{\{0\}}$ with $S(g) = \emptyset$ are all-large items.
Let $|M_{\{0\}}| = 4a + b$ with $b \in \{0, 1, 2, 3\}$.
As mentioned before, we will make sure each agent receives either a items or $a+1$ items.

Fix quotas $q_i \in \{a, a+1\}$ with $\sum_i q_i = |M_{\{0\}}|$.
We consider allocations of $M_{\{0\}}$ with quotas $q = (q_1, q_2, q_3, q_4)$ in which each agent i first receives as many of her
 $\hookrightarrow$ uniquely-small items as possible, up to quota q_i , and the remaining slots are filled with the remaining items.
We will show that all such allocations satisfy EFX.
It is also important that we have the freedom to choose which agents receive a items and which agents receive $a+1$ items.
This freedom enables us to append the allocation of M_2 after the allocation of $M_{\{0\}}$.

We begin by defining this type of allocations.

[Next verbatim source excerpt]

```
\subsection{The  $M_2$  Multigraph Algorithm}
\label{subsec:2}
We now give a complete allocation procedure for the items in  $M_2$ .
Each item is small for some agents  $i$  and  $j$  and
large for the other two agents, and we say that the type of this item is  $(i, j)$ , or  $ij$  for short.
In this section, we consider the case where the instance only has  $M_{\{2\}}$  items.
We represent the instance by a
multigraph  $R$  on  $N = \{1, 2, 3, 4\}$ , where an item of type  $(i, j)$  is an edge  $(i, j)$ .
We slightly abuse the notation by letting  $R$  also represent the edge set, i.e.,  $R$  exactly corresponds to items in  $M_2$ .
Let  $x_{ij}$  denote the multiplicity of edge  $(i, j)$  and let  $m_R = |R|$ .
```

For $U \subseteq N$, define

```
\[
  \operatorname{inc}_R(U) = \left| \{e \in R : e \cap U \neq \emptyset\} \right|
\]
```

and

```
\[
  \Delta_R(U) = \frac{|U| m_R}{4} - \operatorname{inc}_R(U).
\]
```

We call U **deficient** if $\Delta_R(U) > 0$.

[Next verbatim source excerpt]

```
\subsection{Concatenating the  $M_{\{0\}}$  prefix and the  $M_2$  allocation}
\label{subsec:combine}
We now finish the proof for  $M_{\{0\}} \cup M_2$ . Write
\[
  |M_{\{0\}}| = 4a + b,
  \quad b \in \{0, 1, 2, 3\}.
\]
```

We discuss the four values of b .
In each case and each of the sub-case, we will discuss how the concatenation is done and show that the resultant allocation is EFX.
We will exploit many features discussed earlier: the freedom to choose those agents that receive one more item when allocating
 $\hookrightarrow M_{\{0\}}$, the freedom to choose envy-free agents suggested in Lemma~\ref{lem:M2properties}, etc.

At a high level, we will try to use some items in M_2 for gap-filling such that the allocation of $M_{\{0\}}$ in the first phase
 $\hookrightarrow$ becomes envy-free after gap-filling.
If the residual of M_2 is not of the two configurations in Lemma~\ref{lem:M2properties}(ii), we can allocate this residue so
 $\hookrightarrow$ that every envy is up to a small item.
Lemma~\ref{lem:composition} then implies the combined allocation is EFX.
The tricky part is to try to make the residue of M_2 after gap-filling avoid the two configurations in
 $\hookrightarrow$ Lemma~\ref{lem:M2properties}(ii).
However, there are hard cases where this is not always possible, and we need to handle these cases separately.

If, after the gap-filling, the residual M_2 items form one of the two configurations in Lemma~\ref{lem:M2properties}(ii), we
 $\hookrightarrow$ will call this configuration **exceptional**.
In an exceptional configuration, one edge (i, j) has multiplicity at least 3, and, other than those (i, j) -items, there can
 $\hookrightarrow$ be at most one other item of type (i, j) where (i, j) is disjoint from (i, j) .
We will call the two agents i, j **exceptional agents**.
By Lemma~\ref{lem:M2properties}(ii), for the two exceptional agents i, j , one of them receives only large item in the residual
 $\hookrightarrow$ of M_2 and envies the other by a large item.
We call this agent the **disadvantageous agent**.
Lemma~\ref{lem:M2properties}(ii) tells us we are free to choose the disadvantageous agent from the two exceptional agents.

Expanded PaperInterface specification

```

∀ (Item : Type) (r : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin 4) Item)
  (prefixChores m2Chores gapChores residueChores : Finset Item) (a : ℕ) (quota : Fin 4 → ℕ)
  (prefixAllocation gap : EconCSLib.FairDivision.Allocation (Fin 4) Item),
  2 < r →
  EconCSLib.FairDivision.IsOneOrRChoreCost cost r →
  Disjoint prefixChores m2Chores →
  (∀ item ∈ m2Chores, EconCSLib.FairDivision.IsSmallForExactlyTwo cost item) →
  Disjoint gapChores residueChores →
  gapChores ∪ residueChores = m2Chores →
  EconCSLib.FairDivision.IsCanonicalSmallChoreAllocation cost prefixChores quota prefixAllocation →
  (∀ item ∈ prefixChores, EconCSLib.FairDivision.IsSmallForAtMostOne cost item) →
  (∀ (agent : Fin 4), quota agent = a ∨ quota agent = a + 1) →
  EconCSLib.FairDivision.IsAllocationOf gap gapChores →
  (∀ (agent : Fin 4), quota agent ≠ a → gap agent = ∅) →
  (EconCSLib.FairDivision.EnvyFreeForChores (EconCSLib.FairDivision.additiveChoreCost cost)
    fun agent => prefixAllocation agent ∪ gap agent) →
  ∀ (exceptionalI exceptionalJ : Fin 4),
    (∃ dominant auxiliary q,
      dominant.card = 2 ∧
      auxiliary.card = 2 ∧
      Disjoint dominant auxiliary ∧
      exceptionalI ∈ auxiliary ∧
      exceptionalJ ∈ auxiliary ∧
      ((∀ item ∈ residueChores,
        EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
        residueChores.card = 4 * q + 3 ∨
        ∃ exceptionalItem ∈ residueChores,
        EconCSLib.FairDivision.smallAgentSet cost exceptionalItem = auxiliary ∧
        (∀ item ∈ residueChores.erase exceptionalItem,
          EconCSLib.FairDivision.smallAgentSet cost item = dominant) ∧
        (residueChores.erase exceptionalItem).card = 4 * q + 3)) →
    exceptionalI ≠ exceptionalJ →
    quota exceptionalI = a + 1 →
    quota exceptionalJ = a + 1 →
    ∃ allocation,
      EconCSLib.FairDivision.IsAllocationOf allocation (prefixChores ∪ m2Chores) ∧
      EconCSLib.FairDivision.EFXForChores
        (EconCSLib.FairDivision.additiveChoreCost cost) allocation

```

Lean proof endpoint (built separately)

HT26EFXChores.exceptional_residue_combination

Recorded source-to-Spec screening Verdict: matches

Reason: The raw lemma, canonical-definition context, M2 definition, and exceptional/gap-filling construction describe a canonical M01 prefix extended to envy-freeness and an exceptional M2 residue with two long exceptional agents. The transparent Spec makes precisely those pool, allocation, quota, exceptional-agent, and EFX conclusion conditions explicit.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix and supplement source claims

Review row 21

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:2074-2076

Verbatim source input

```
\begin{proposition}
  In an EFX allocation, no bundle contains two or more items from  $A$ .
\end{proposition}

[Next verbatim source excerpt]

In this section, we prove Theorem~\ref{thm:tri} for an arbitrary fixed  $n \geq 4$ .

\subsection{The construction}
Fix an integer  $n \geq 4$ . Partition the agents into two groups  $T_1$  and  $T_2$  such that
\[\begin{aligned} |T_1| &= \left\lfloor \frac{n}{2} \right\rfloor, \\ |T_2| &= \left\lceil \frac{n}{2} \right\rceil. \end{aligned}\]
Let  $t_1 = |T_1|$  and  $t_2 = |T_2|$ .
Let  $s = 2t_2 + 1$ .
The instance contains  $n - 1 + 2s$  items that are partitioned into three groups  $A, B, C$  where  $A$  contains  $n - 1$  items and each of
 $\hookrightarrow B$  and  $C$  contains  $s$  items.

The three values in the tri-valued instances are given by

$$p = 1, \quad q = s + 2t_2 + 3, \quad \text{and} \quad r = \frac{(s + 1)^2}{2} = (2t_2 + 1)(t_2 + 2).$$


The costs are defined as follows. For every agent  $i \in T_1$ ,
\[\begin{aligned} c_i(a) &= r \\ c_i(b) &= p \\ c_i(c) &= q \end{aligned}\]
and for every agent  $i \in T_2$ ,
\[\begin{aligned} c_i(a) &= r \\ c_i(b) &= q \\ c_i(c) &= p \end{aligned}\]

For  $n = 4$ , we have  $t_1 = t_2 = 2$ ,  $s = 5$ ,  $q = 7$ , and  $r = 20$ .
This resembles our construction in Sect.~\ref{sec:tri}.

\subsection{Non-existence of EFX allocations}
Let  $X = (X_1, \dots, X_n)$  be an arbitrary allocation. For a bundle  $Y \subseteq A \cup B \cup C$ , write
\[\begin{aligned} a(Y) &= |Y \cap A|, \\ b(Y) &= |Y \cap B|, \\ c(Y) &= |Y \cap C|. \end{aligned}\]
All agents in  $T_1$  have the same cost for a bundle  $Y$ , namely
\[\begin{aligned} P_1(Y) &= r \cdot a(Y) + b(Y) + q \cdot c(Y), \end{aligned}\]
and all agents in  $T_2$  have the same cost for  $Y$ , namely
\[\begin{aligned} P_2(Y) &= r \cdot a(Y) + q \cdot b(Y) + c(Y). \end{aligned}\]
Define
\[\begin{aligned} p_1 &= \min_{i \in \{1, \dots, n\}} P_1(X_i), \\ p_2 &= \min_{i \in \{1, \dots, n\}} P_2(X_i). \end{aligned}\]
```

Expanded PaperInterface specification

```
∀ (n : ℕ) (Item : Type) (r q : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin n) Item) (chores A B C : Finset Item)
  (allocation : EconCSLib.FairDivision.Allocation (Fin n) Item),
  4 ≤ n →
  q = ⌈(2 * ((n + 1) / 2) + 1) + 2⌉ →
  r = ⌈(2 * ((n + 1) / 2) + 1) * (q + 1) / 2⌉ →
  A.card = n - 1 →
  B.card = 2 * ((n + 1) / 2) + 1 →
  C.card = 2 * ((n + 1) / 2) + 1 →
  Disjoint A B →
  Disjoint A C →
  Disjoint B C →
  A ∪ B ∪ C = chores →
  (∀ (agent : Fin n) (item : Item),
    (item ∈ A → cost agent item = r) ∧
    (item ∈ B → cost agent item = if ragent < n / 2 then 1 else q) ∧
```

```

      (item ∈ C → cost agent item = if ↑agent < n / 2 then q else 1)) →
EconCSLib.FairDivision.IsAllocationOf allocation chores →
EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost)
allocation →
∀ (agent : Fin n), Finset.card (allocation agent n A) ≤ 1

```

Lean proof endpoint (built separately)

HT26EFXChores.appendix_no_two_a_items

Recorded source-to-Spec screening Verdict: matches

Reason: The raw Appendix-A construction fixes n , the two groups, the A/B/C cardinalities, $p=1$, q , r , and the complete cost table; the proposition says no EFX bundle has two A items. The transparent Spec encodes that same finite construction and concludes that every bundle has at most one A item.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 22

Source locator: papers/HT26EFXChores/source/EFXadditivechores.tex:2104-2106

Verbatim source input

```
\begin{proposition}
  In an EFX allocation,  $p_1 \geq r$  and  $p_2 \geq r$ .
\end{proposition}

[Next verbatim source excerpt]

In this section, we prove Theorem~\ref{thm:tri} for an arbitrary fixed  $n \geq 4$ .

\subsection{The construction}
Fix an integer  $n \geq 4$ . Partition the agents into two groups  $T_1$  and  $T_2$  such that
\[\begin{aligned} |T_1| &= \left\lfloor \frac{n}{2} \right\rfloor, \\ |T_2| &= \left\lceil \frac{n}{2} \right\rceil. \end{aligned}\]
Let  $t_1 = |T_1|$  and  $t_2 = |T_2|$ .
Let  $s = 2t_1 + 1$ .
The instance contains  $n-1+2s$  items that are partitioned into three groups  $A, B, C$  where  $A$  contains  $n-1$  items and each of
 $\hookrightarrow B$  and  $C$  contains  $s$  items.

The three values in the tri-valued instances are given by

$$p=1, \quad q=s+2t_2+3, \quad r=\frac{(s+1)^2}{2} = (t_2+1)(t_2+2).$$


The costs are defined as follows. For every agent  $i \in T_1$ ,
\[\begin{aligned} c_i(a) &= r \\ c_i(b) &= p \\ c_i(c) &= q \end{aligned}\]
and for every agent  $i \in T_2$ ,
\[\begin{aligned} c_i(a) &= r \\ c_i(b) &= q \\ c_i(c) &= p \end{aligned}\]

For  $n=4$ , we have  $t_1=t_2=2$ ,  $s=5$ ,  $q=7$ , and  $r=20$ .
This resembles our construction in Sect.~\ref{sec:tri}.

\subsection{Non-existence of EFX allocations}
Let  $X=(X_1, \dots, X_n)$  be an arbitrary allocation. For a bundle  $Y \subseteq A \cup B \cup C$ , write
\[\begin{aligned} a(Y) &= |Y \cap A|, \\ b(Y) &= |Y \cap B|, \\ c(Y) &= |Y \cap C|. \end{aligned}\]
All agents in  $T_1$  have the same cost for a bundle  $Y$ , namely
\[\begin{aligned} P_1(Y) &= r \cdot a(Y) + b(Y) + q \cdot c(Y), \end{aligned}\]
and all agents in  $T_2$  have the same cost for  $Y$ , namely
\[\begin{aligned} P_2(Y) &= r \cdot a(Y) + q \cdot b(Y) + c(Y). \end{aligned}\]
Define
\[\begin{aligned} p_1 &= \min_{i \in \{1, \dots, n\}} P_1(X_i), \\ p_2 &= \min_{i \in \{1, \dots, n\}} P_2(X_i). \end{aligned}\]
```

Expanded PaperInterface specification

```
∀ (n : ℕ) (Item : Type) (r q : ℝ) (cost : EconCSLib.FairDivision.ChoreCost (Fin n) Item) (chores A B C : Finset Item)
  (allocation : EconCSLib.FairDivision.Allocation (Fin n) Item) (P1 P2 : EconCSLib.FairDivision.Bundle Item → ℝ)
  (p1 p2 : ℝ),
  4 ≤ n →
  q = ⌈2 * ((n + 1) / 2) + 1⌉ + 2 →
  r = ⌈2 * ((n + 1) / 2) + 1⌉ * (q + 1) / 2 →
  A.card = n - 1 →
  B.card = 2 * ((n + 1) / 2) + 1 →
  C.card = 2 * ((n + 1) / 2) + 1 →
  Disjoint A B →
  Disjoint A C →
  Disjoint B C →
  A ∪ B ∪ C = chores →
  (∀ (agent : Fin n) (item : Item),
    (item ∈ A → cost agent item = r) ∧
    (item ∈ B → cost agent item = if agent < n / 2 then 1 else q) ∧
    (item ∈ C → cost agent item = if agent < n / 2 then q else 1)) →
  EconCSLib.FairDivision.IsAllocationOf allocation chores →
```



```

EconCSLib.FairDivision.EFXForChores (EconCSLib.FairDivision.additiveChoreCost cost)
allocation →
(∀ (bundle : EconCSLib.FairDivision.Bundle Item),
  P1 bundle =
    r * ↑(Finset.card (bundle n A)) + ↑(Finset.card (bundle n B)) +
    q * ↑(Finset.card (bundle n C))) →
(∀ (bundle : EconCSLib.FairDivision.Bundle Item),
  P2 bundle =
    r * ↑(Finset.card (bundle n A)) + q * ↑(Finset.card (bundle n B)) +
    ↑(Finset.card (bundle n C))) →
(∀ (agent : Fin n),
  ↑agent < n / 2 → EconCSLib.FairDivision.additiveChoreCost cost agent = P1) →
(∀ (agent : Fin n),
  n / 2 ≤ ↑agent → EconCSLib.FairDivision.additiveChoreCost cost agent = P2) →
(∃ agent, P1 (allocation agent) = p1) →
(∀ (agent : Fin n), p1 ≤ P1 (allocation agent)) →
(∃ agent, P2 (allocation agent) = p2) →
(∀ (agent : Fin n), p2 ≤ P2 (allocation agent)) → p1 ≥ r ∧ p2 ≥ r

```

Lean proof endpoint (built separately)

HT26EFXChores.appendix_p1_p2_lower_bounds

Recorded source-to-Spec screening Verdict: matches

Reason: The raw Appendix-A construction and notation define P1, P2, p1, and p2 before the proposition p1,p2 at least r. The transparent Spec uses those same construction equations, P1/P2 definitions, group-specific costs, and minimum witnesses to conclude both lower bounds.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation